

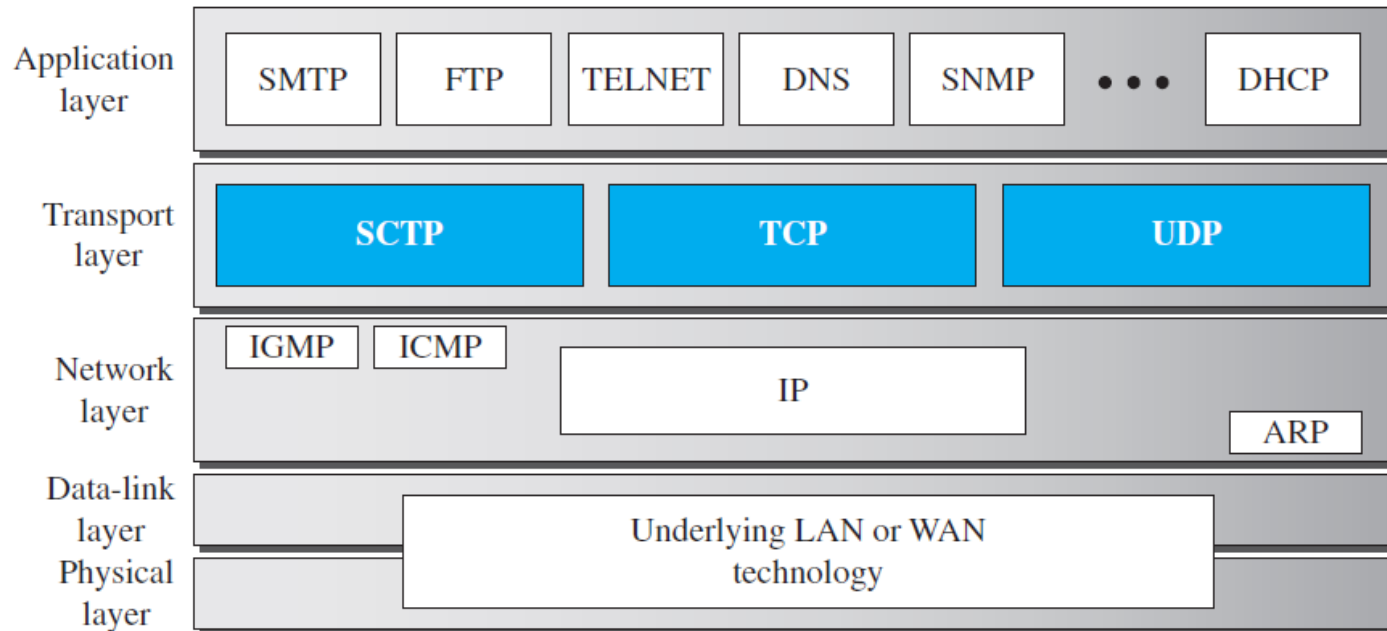
Unit-4

Transport-Layer Protocols

Introduction

- The transport layer acts as a liaison between a client program and a server program, a process-to-process connection

Figure 24.1 *Position of transport-layer protocols in the TCP/IP protocol suite*



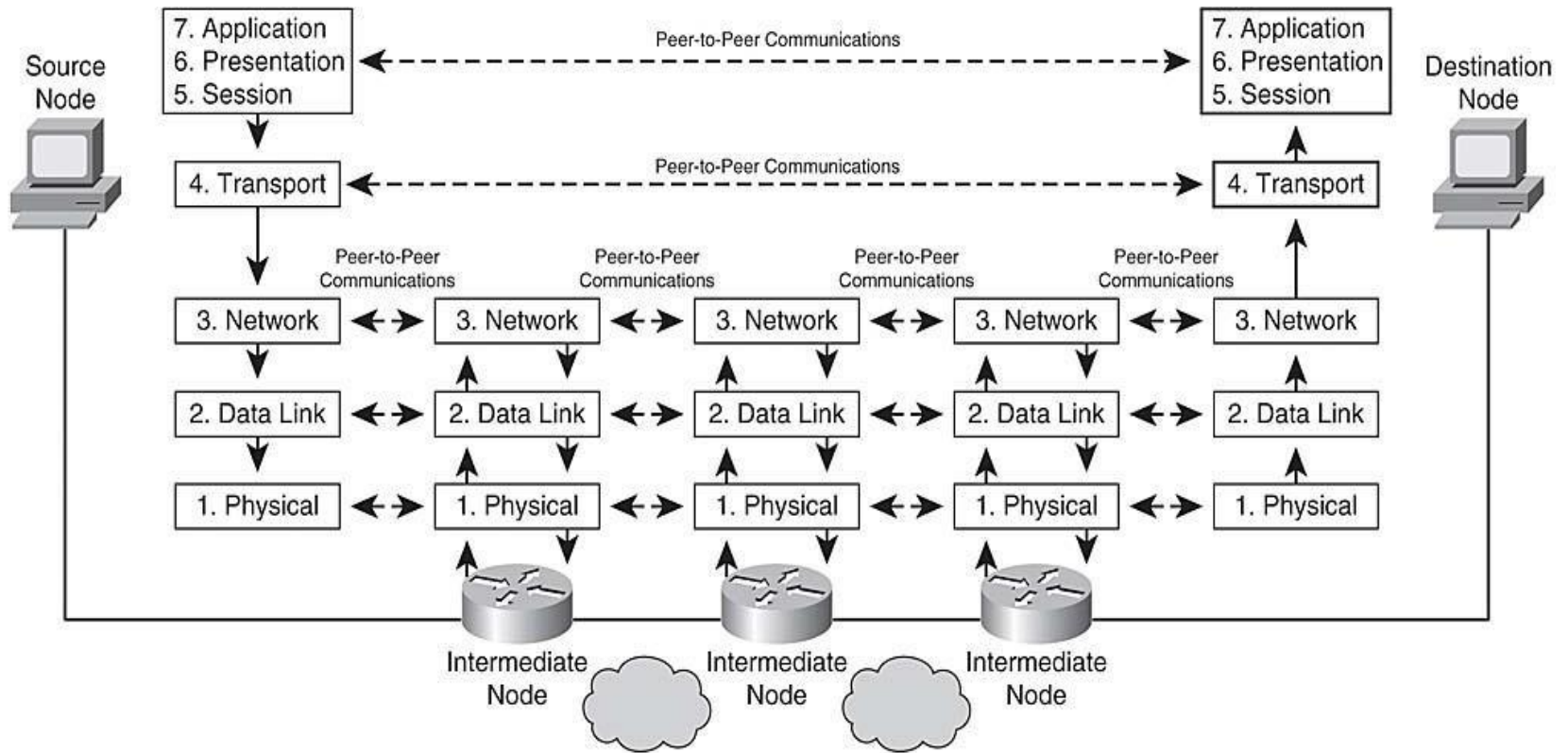
Transport Layer

Services:

- create a process-to-process communication; these protocols use port numbers to accomplish this
 - Service-point addressing (Port address)
 - Connection control (Connectionless or connection oriented)
 - Segmentation and reassembly (Sequence number)
 - Flow control (end to end)
 - Error Control (Process to Process)

Protocols: TCP, UDP, SCTP

Transport Layer



- Communication is provided using a logical connection- imaginary direct connection through which they can send and receive messages
- only the two end systems use the services of the transport layer; all intermediate routers use only the first three layers.

Services

Each protocol provides a different type of service and should be used appropriately

UDP

- UDP is an **unreliable connectionless transport-layer protocol** used for its simplicity and efficiency in applications where error control can be provided by the application-layer process.

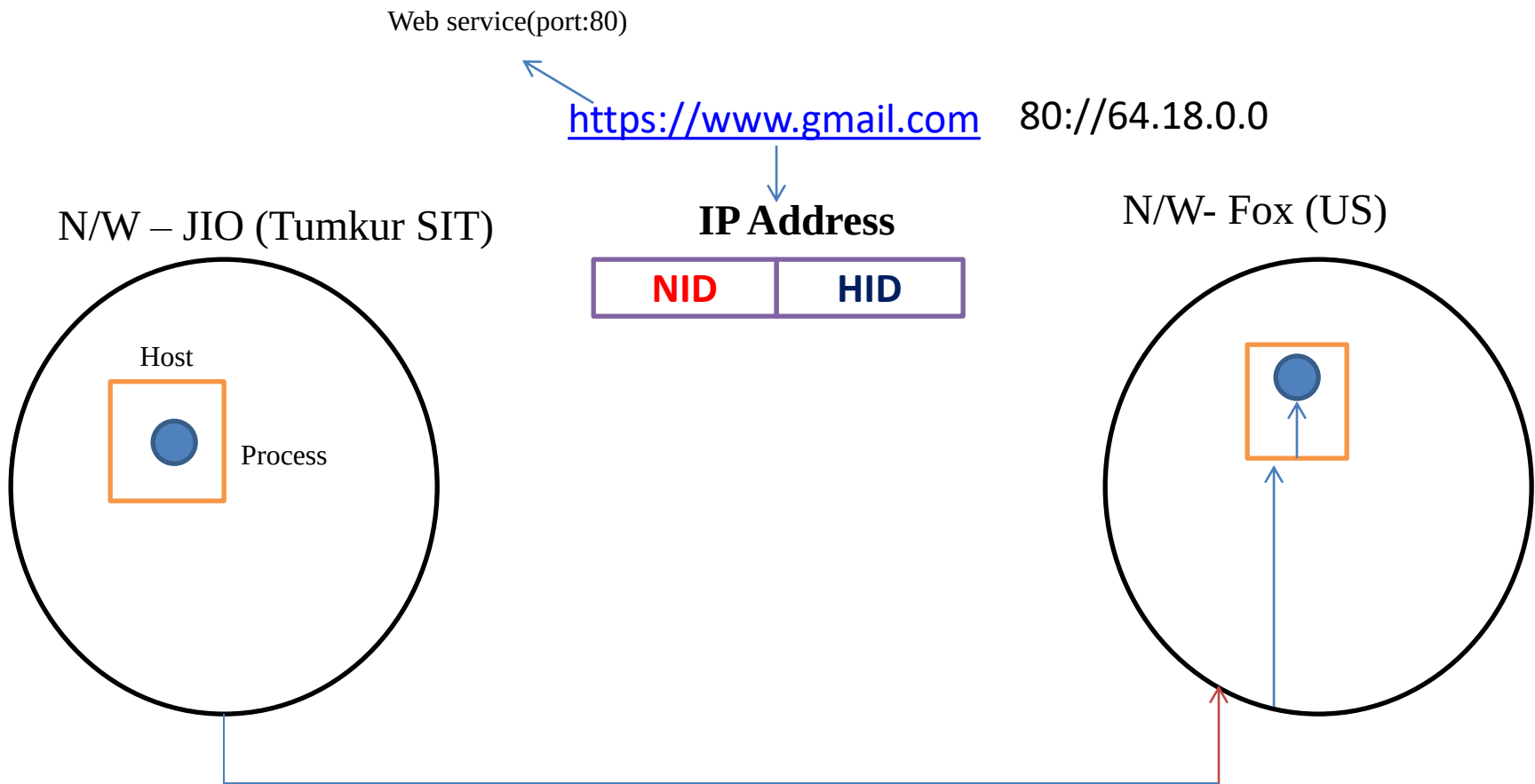
TCP

- TCP is a **reliable connection-oriented protocol** that can be used in any application where reliability is important.

SCTP

- SCTP is a new transport-layer protocol that combines the features of UDP and TCP

Port Numbers



- One is to create a process-to-process communication
- These protocols use port numbers to accomplish this

Port Numbers

Table 24.1 *Some well-known ports used with UDP and TCP*

Port	Protocol	UDP	TCP	SCTP	Description
7	Echo	√	√	√	Echoes back a received datagram
9	Discard	√	√	√	Discards any datagram that is received
11	Users	√	√	√	Active users
13	Daytime	√	√	√	Returns the date and the time
17	Quote	√	√	√	Returns a quote of the day
19	Chargen	√	√	√	Returns a string of characters
20	FTP-data		√	√	File Transfer Protocol
21	FTP-21		√	√	File Transfer Protocol
23	TELNET		√	√	Terminal Network
25	SMTP		√	√	Simple Mail Transfer Protocol
53	DNS	√	√	√	Domain Name Service
67	DHCP	√	√	√	Dynamic Host Configuration Protocol
69	TFTP	√	√	√	Trivial File Transfer Protocol
80	HTTP		√	√	HyperText Transfer Protocol
111	RPC	√	√	√	Remote Procedure Call
123	NTP	√	√	√	Network Time Protocol
161	SNMP-server	√			Simple Network Management Protocol
162	SNMP-client	√			Simple Network Management Protocol

USER DATAGRAM PROTOCOL

- The **User Datagram Protocol (UDP)** is a **connectionless, unreliable transport protocol**.
- UDP is a very simple protocol using a minimum of overhead.
 - If a process wants to send a small message and does not care much about reliability
- UDP takes much less interaction between the sender and receiver.

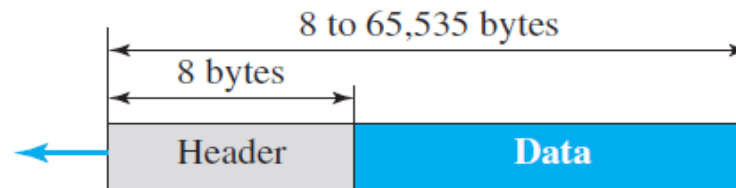
Applications of UDP

- Domain Name System (DNS)
- Streaming media applications IPTV, i.e. watching TV on a computer system.
- SNMP
- DHCP
- VOIP

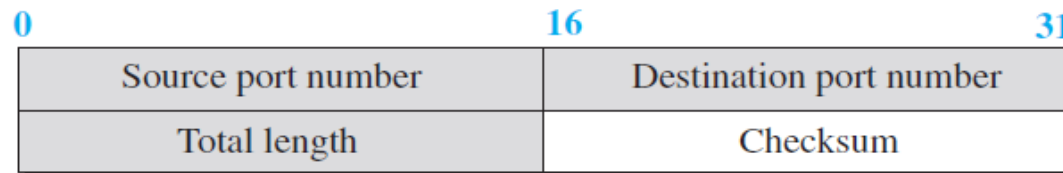
User Datagram

- UDP packets, called *user datagrams*
- The first two fields define the source and destination port numbers
- The third field defines the total length of the user datagram, header plus data (16 bits-0 to 65,535 bytes)
- The last field can carry the optional checksum

Figure 24.2 *User datagram packet format*



a. UDP user datagram



b. Header format

Example

- The following is the content of a UDP header in hexadecimal format : **CB84000D001C001C**.
 - a. What is the source port number?
 - b. What is the destination port number?
 - c. What is the total length of the user datagram?
 - d. What is the length of the data?
 - e. Is the packet directed from a client to a server or vice versa?
 - f. What is the client process?

Example

- The following is the content of a UDP header in hexadecimal format : **CB84000D001C001C**.

a. What is the source port number? **(CB84)₁₆ = 52100**

b. What is the destination port number? **(000D)₁₆ = 13**

c. What is the total length of the user datagram?

001C = whole UDP packet as 28 bytes.

d. What is the length of the data?

28 – 8 = 20 bytes

e. Is the packet directed from a client to a server or vice versa?

- Since the destination port number is 13 (well-known port), the packet is from the client to the server.

f. What is the client process?

The client process is the Daytime

UDP Services

- *Process-to-Process Communication.*
- *Connectionless Services*
- *Flow Control*
- *Error Control*
- *Checksum*
- *Congestion Control*
- *Encapsulation and Decapsulation.*
- *Queuing*
- *Multiplexing and Demultiplexing*

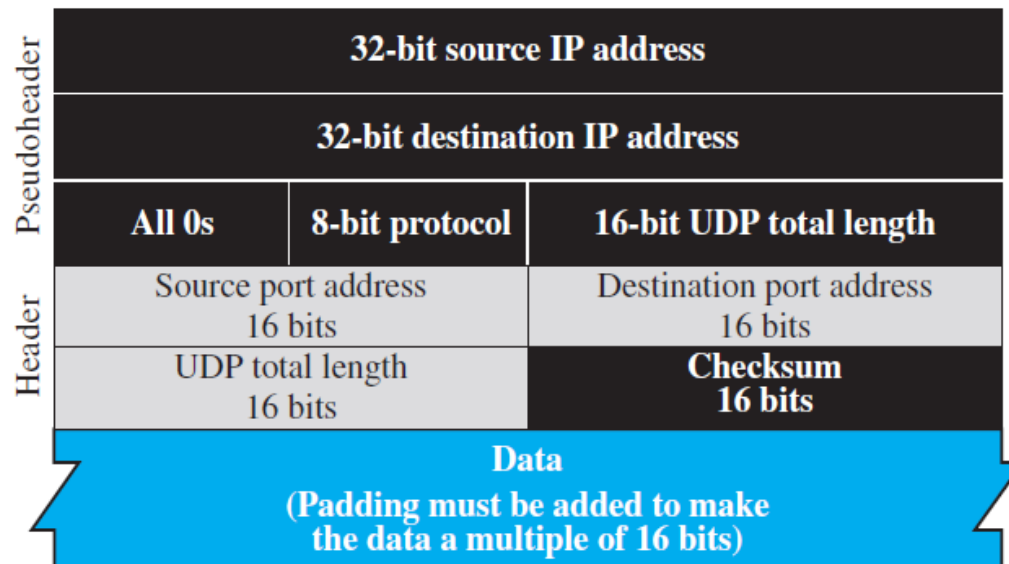
UDP Services

- **Process-to-Process Communication:** Using socket addresses, a combination of IP addresses and port numbers.
- **Connectionless Services:** each user datagram sent by UDP is an independent datagram
 - There is no relationship between the different user datagrams even if they are coming from the same source process and going to the same destination program.
 - The user datagrams are not numbered. each request must be small enough to fit into one user datagram
- **Flow Control:** There is no *flow control*, and hence no window mechanism.
- **Error Control:** no *error control* mechanism in UDP except for the checksum
 - the sender does not know if a message has been lost or duplicated
 - When the receiver detects an error through the checksum, the user datagram is silently discarded
- **Checksum:** UDP checksum calculation includes three sections: a pseudoheader, the UDP header, and the data coming from the application layer.

Checksum:

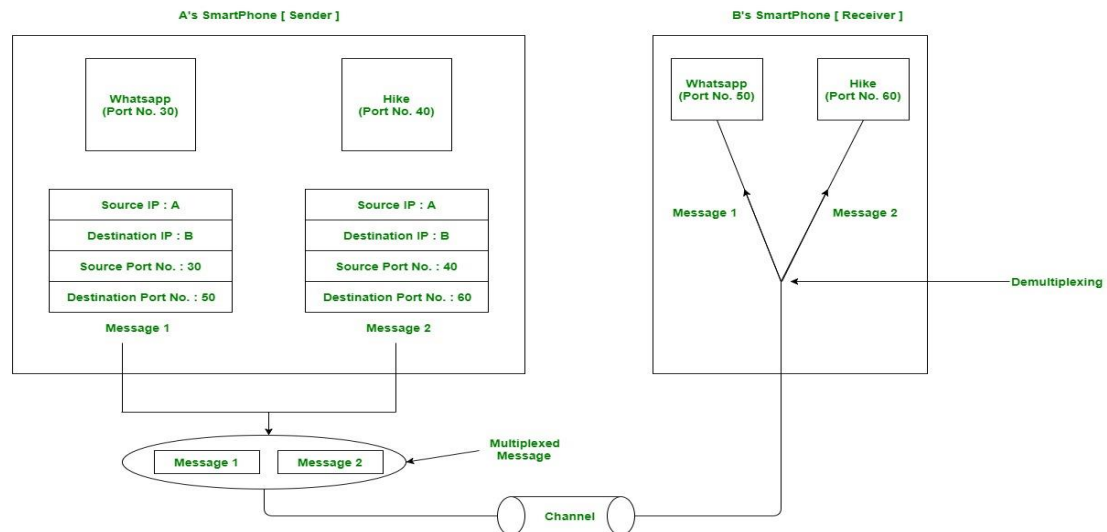
- The *pseudoheader* is the part of the header of the IP packet
- if the IP header is corrupted, it may be delivered to the wrong host – hence including the pseudoheader
- The protocol field is added to ensure that the packet belongs to UDP, and not to TCP.
- The value of the protocol field for UDP is 17

Figure 24.3 *Pseudoheader for checksum calculation*



UDP Services

- **Congestion Control:** UDP does not provide congestion control
 - UDP assumes that the packets sent are small and sporadic and cannot create congestion in the network.
- **Encapsulation and Decapsulation:** To send a message from one process to another, the UDP protocol encapsulates and decapsulates messages
- **Queuing:** At the client site, when a process starts, it requests a port number from the operating system. Some implementations create both an incoming and an outgoing queue associated with each process.
- **Multiplexing and Demultiplexing:** Several processes that may want to use the services of UDP. To handle this situation, UDP multiplexes and demultiplexes.



UDP – Connectionless, no flow control, no error,
and no congestion control!

WHY WOULD ANYONE USE UDP

UDP is light weight protocol :

- **No delay for connection establishment**
 - UDP just blasts away without any formal preliminaries
 - ... which avoids introducing any unnecessary delays
- **No connection state**
 - No allocation of buffers, parameters, sequence #s, etc.
 - ... making it easier to handle many active clients at once
- **Small packet header overhead**
 - UDP header is only eight-bytes long



UDP Applications

Connectionless Service: This feature can be considered as an advantage or disadvantage depending on the application requirements.

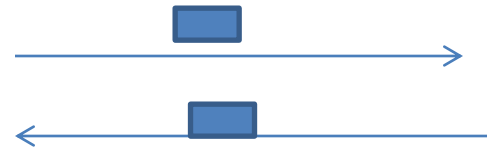
For example, a client application needs to send a short request to a server and to receive a short response.

- The overhead to establish and close a connection is less
- Connection oriented service - at least 9 packets are exchanged between the client and the server
- In connectionless service - only 2 packets are exchanged
- If delay is an important issue for the application, the connectionless service is preferred.
- If the request and response can each fit in a single user datagram the connectionless service is preferred.

Examples – connectionless oriented

Example 24.3

A client-server application such as DNS (see Chapter 26) uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it. The request and response can each fit in one user datagram. Since only one message is exchanged in each direction, the connectionless feature is not an issue; the client or server does not worry that messages are delivered out of order.



Example 24.4

A client-server application such as SMTP (see Chapter 27), which is used in electronic mail, cannot use the services of UDP because a user might send a long e-mail message, which could include multimedia (images, audio, or video). If the application uses UDP and the message does not fit in one user datagram, the message must be split by the application into different user datagrams. Here the connectionless service may create problems. The user datagrams may arrive and be delivered to the receiver application out of order. The receiver application may not be able to reorder the pieces. This means the connectionless service has a disadvantage for an application program that sends long messages. In SMTP, when we send a message, we do not expect to receive a response quickly (sometimes no response is required). This means that the extra delay inherent in connection-oriented service is not crucial for SMTP.

UDP Applications

***Lack of Error Control:**UDP does not provide error control; it provides an unreliable service. Some applications, by nature, do not even notice these uneven delays, but for some they are very problematic*

Example 24.5

Assume we are downloading a very large text file from the Internet. We definitely need to use a transport layer that provides reliable service. We don't want part of the file to be missing or corrupted when we open the file. The delay created between the deliveries of the parts is not an overriding concern for us; we wait until the whole file is composed before looking at it. In this case, UDP is not a suitable transport layer.

Example 24.6

Assume we are using a real-time interactive application, such as Skype. Audio and video are divided into frames and sent one after another. If the transport layer is supposed to resend a corrupted or lost frame, the synchronizing of the whole transmission may be lost. The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable. However, if each small part of the screen is sent using a single user datagram, the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program. That part of the screen is blank for a very short period of time, which most viewers do not even notice.

Typical Applications

- UDP is suitable for a process that requires **simple request-response communication** with little concern for flow and error control
- UDP is a suitable transport protocol for **multicasting**
- UDP is used for management processes such as **SNMP**
- UDP is used for some route updating protocols such as **Routing Information Protocol (RIP)**
- UDP is normally used for interactive **real-time applications** that cannot tolerate uneven delay between sections of a received message

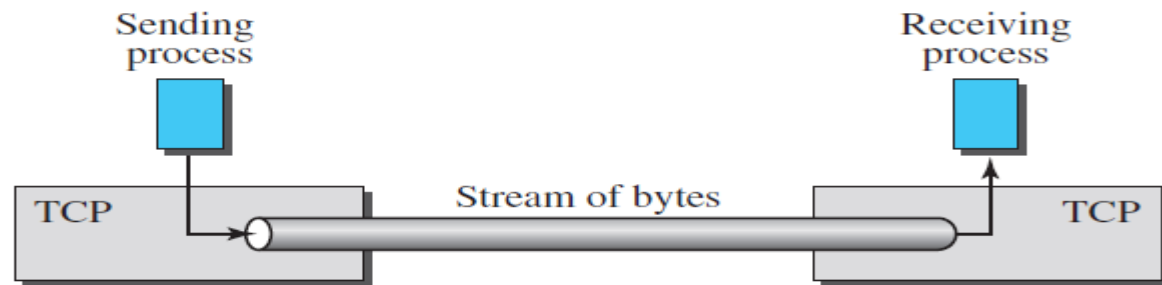
TRANSMISSION CONTROL PROTOCOL

- **Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol.**
- TCP explicitly defines connection establishment, data transfer, and connection teardown **phases** to provide a connection-oriented service.
- TCP uses checksum (for error detection), retransmission of lost or corrupted packets, cumulative and selective acknowledgments, and timers.
- TCP is the most common transport-layer protocol in the Internet.

TCP Services

- **Process-to-Process Communication:** TCP provides process-to-process communication using port numbers
- **Stream Delivery Service:** TCP allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes. TCP creates an environment in which the two processes seem to be connected by an imaginary “tube” that carries their bytes across the Internet.

Figure 24.4 *Stream delivery*



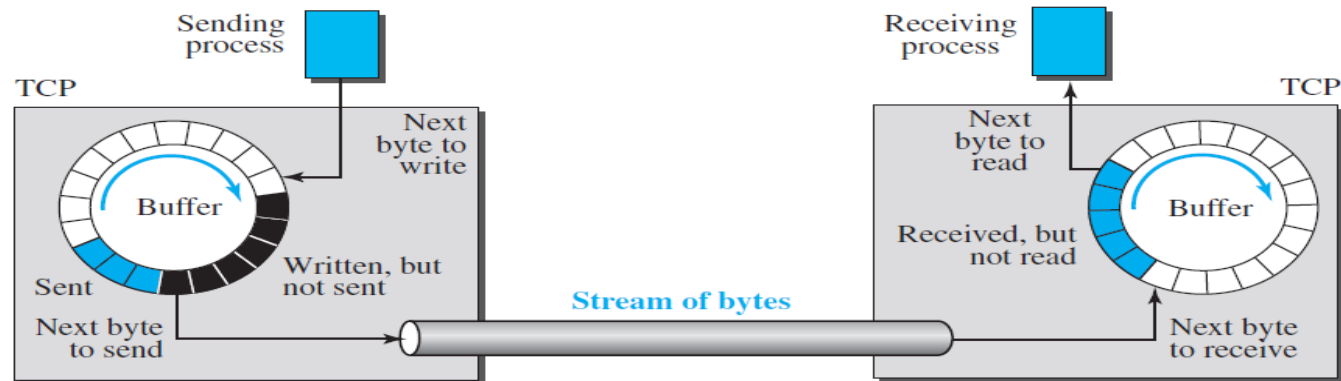
The sending process produces (writes to) the stream and the receiving process consumes (reads from) it.

TCP Services

Sending and Receiving Buffers: TCP needs buffers for storage. There are two buffers, the sending buffer and the receiving buffer, one for each direction

- Buffers are also necessary for flow- and error-control mechanisms used by TCP.
- One way to implement a buffer is to use a circular array of 1-byte locations

Figure 24.5 *Sending and receiving buffers*



The buffer has three types of chambers:

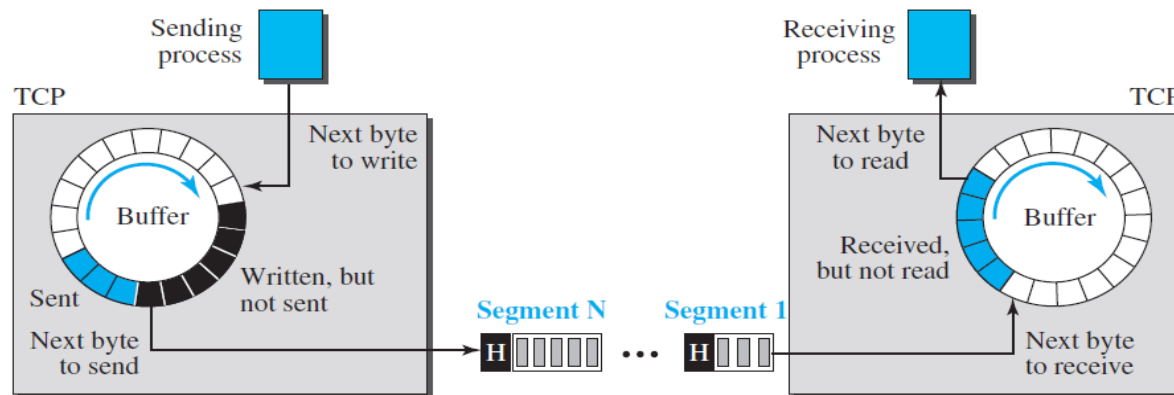
- white section contains empty chambers that can be filled by the sending process
- colored area holds bytes that have been sent but not yet acknowledged
- shaded area contains bytes to be sent by the sending TCP

TCP Services

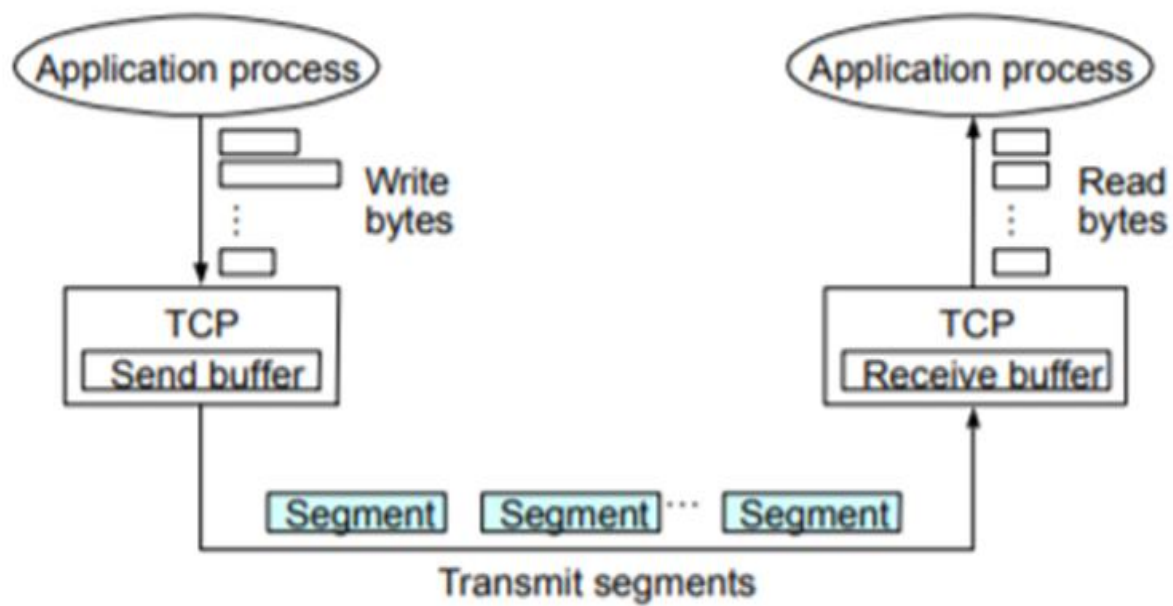
Segments

- we need one more step before we can send data
- At the transport layer, **TCP groups a number of bytes together** into a packet called a *segment*
- TCP adds a header to each segment (for control purposes) and delivers the segment to the network layer for transmission.
- The segments are encapsulated in an IP datagram and transmitted.

Figure 24.6 TCP segments



Full-Duplex Communication: TCP offers *full-duplex service*, where data can flow in both directions at the same time. segments move in both directions.



TCP Services

- ***Multiplexing and Demultiplexing:*** Like UDP, TCP performs multiplexing at the sender and demultiplexing at the receiver.
- ***Connection-Oriented Service:*** When a process at site A wants to send to and receive data from another process at site B, the following three phases occur:
 1. The two TCP's establish a logical connection between them.
 2. Data are exchanged in both directions.
 3. The connection is terminated.
- The TCP segment is encapsulated in an IP datagram and can be sent out of order, or lost or corrupted, and then resent. Each may be routed over a different path to reach the destination. There is no physical connection.
- ***Reliable Service:*** TCP is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data.

TCP Features

To provide the services mentioned in the previous section, TCP has several features that are briefly summarized in this section

Numbering System

- TCP software keeps track of the segments being transmitted or received
- There are two fields, called the *sequence number* and the *acknowledgment number* - These two fields refer to a **byte number** and not a segment number.

Byte Number

- When TCP receives bytes of data from a process, TCP stores them in the sending buffer and numbers them.
- The numbering does not necessarily start from 0. Instead, TCP chooses an arbitrary number between 0 and $2^{32} - 1$ for the number of the first byte
- Ex: if the number happens to be 1057 and the total data to be sent is 6000 bytes, the bytes are numbered from 1057 to 7056.

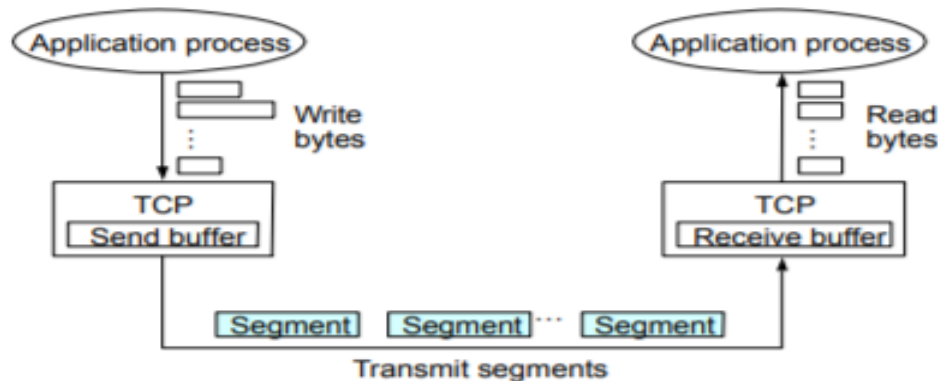
**The bytes of data being transferred in each connection are numbered by TCP.
The numbering starts with an arbitrarily generated number.**

TCP Features

Sequence Number: After the bytes have been numbered, TCP assigns a sequence number to each segment that is being sent. The sequence number, in each direction, is defined as follows:

1. The sequence number of the first segment is the ISN (initial sequence number), which is a random number.
2. The sequence number of any other segment is the sequence number of the previous segment plus the number of bytes (real or imaginary) carried by the previous segment.

Segment N
- H 0 0 0 0 0 0



Example 24.7

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10001	Range:	10001	to	11000
Segment 2	→	Sequence Number:	11001	Range:	11001	to	12000
Segment 3	→	Sequence Number:	12001	Range:	12001	to	13000
Segment 4	→	Sequence Number:	13001	Range:	13001	to	14000
Segment 5	→	Sequence Number:	14001	Range:	14001	to	15000

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

$$10001 + 1000 = 11001 \quad 11001 + 1000 = 12001 \quad 12001 + 1000 = 13001 \quad 13001 + 1000 = 14001$$

TCP Features

Acknowledgment Number

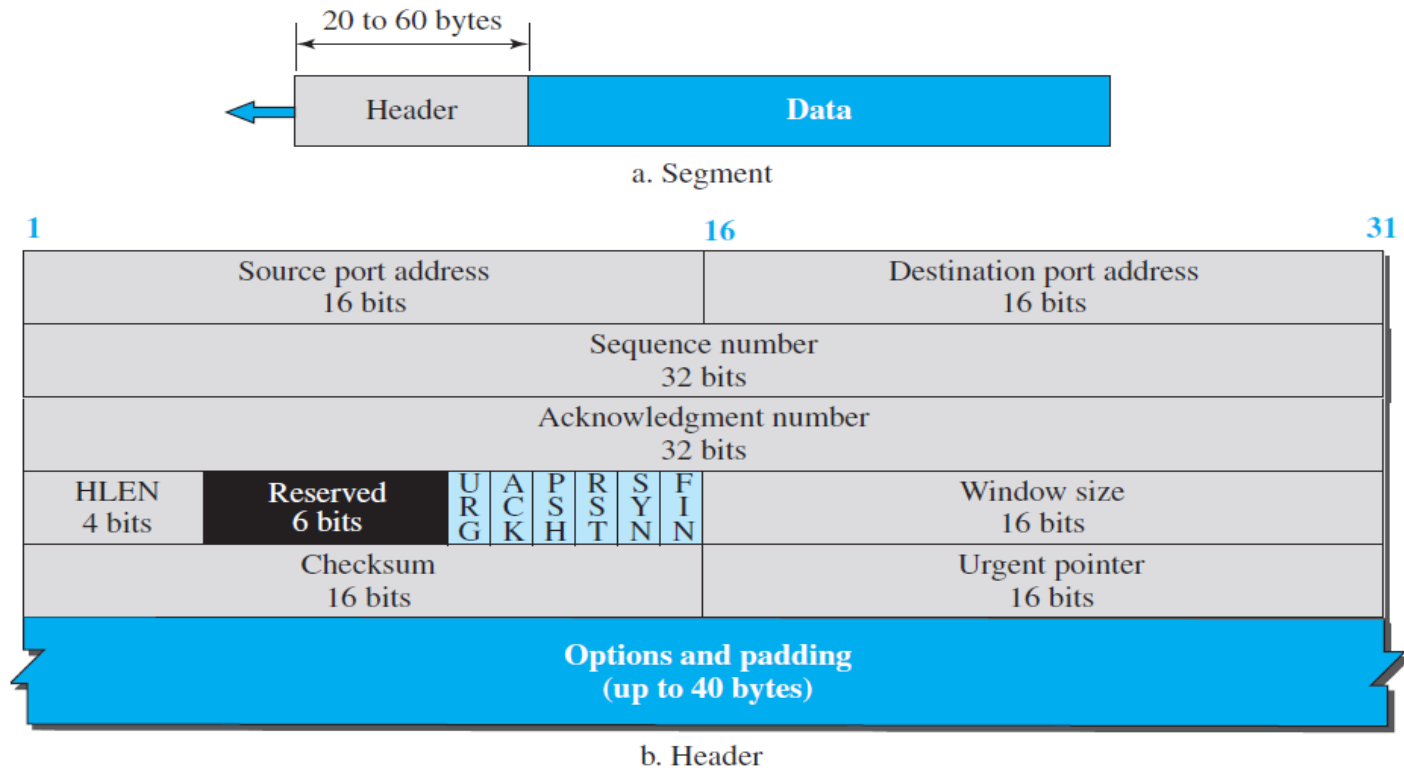
- Each party also uses an acknowledgment number to confirm the bytes it has received
- The acknowledgment number defines the number of the next byte that the party expects to receive.
- In addition, the acknowledgment number is cumulative, which means that the party takes the number of the last byte that it has received, safe and sound, adds 1 to it, and announces this sum as the acknowledgment number.
- The term *cumulative* here means that if a party uses 5643 as an acknowledgment number, it has received all bytes from the beginning up to 5642. Note that this does not mean that the party has received 5642 bytes, because the first byte number does not have to be 0.

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive. The acknowledgment number is cumulative.

Segment

- A packet in TCP is called a segment.

Figure 24.7 *TCP segment format*



The TCP Segment Header

1. Source Port-

Source Port is a 16 bit field.

It identifies the port of the sending application.

2. Destination Port-

Destination Port is a 16 bit field.

It identifies the port of the receiving application.

NOTE

- A TCP connection is uniquely identified by using
 - Combination of port numbers and IP Addresses of sender and receiver
- IP Addresses indicate which systems are communicating.
- Port numbers indicate which end to end sockets are communicating.

The TCP Segment Header

3. Sequence Number-

- Sequence number is a 32 bit field.
- TCP assigns a unique sequence number to each byte of data contained in the TCP segment.
- This field contains the sequence number of the first data byte.

4. Acknowledgement Number-

- Acknowledgment number is a 32 bit field.
- It contains sequence number of the data byte that receiver expects to receive next from the sender.
- It is always sequence number of the last received data byte incremented by 1.

The TCP Segment Header

5. Header Length-

- Header length is a 4 bit field. So, the range of decimal values that can be represented is [0, 15].
- The length of TCP header always lies in the range- [20 bytes , 60 bytes]
- So, to represent the header length, we use a scaling factor of 4.
- Header length = Header length field value x 4 bytes

 If header length field contains decimal value 5 (represented as 0101), then-

 Header length = $5 \times 4 = 20$ bytes

 If header length field contains decimal value 10 (represented as 1010), then-

 Header length = $10 \times 4 = 40$ bytes

 If header length field contains decimal value 15 (represented as 1111), then-

 Header length = $15 \times 4 = 60$ bytes

The TCP Segment Header

6. Reserved Bits-

- The 6 bits are reserved.
- These bits are not used.

7. Window Size-

- Window size is a 16 bit field.

This value is normally referred to as the receiving window (*rwnd*) and is determined by the receiver. The sender must obey the dictation of the receiver in this case..

- Thus, window size is used for [Flow Control](#).

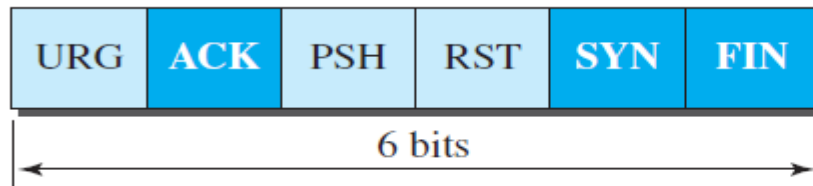
NOTE

It is important to note-

- The window size changes dynamically during data transmission.
- It usually increases during TCP transmission up to a point where congestion is detected.
- After congestion is detected, the window size is reduced to avoid having to drop packets.

8. Control: These bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP.

Figure 24.8 *Control field*



URG: Urgent pointer is valid

ACK: Acknowledgment is valid

PSH : Request for push

RST : Reset the connection

SYN: Synchronize sequence numbers

FIN : Terminate the connection

Wireshark packet capture showing a 3-way TCP handshake. The first packet is a SYN from 10.0.2.15 to 198.57.208.223. The second packet is a SYN, ACK from 198.57.208.223 to 10.0.2.15. The third packet is an ACK from 10.0.2.15 to 198.57.208.223. A blue circle highlights these three packets, with a handwritten note "3 WAY HANDSHAKE" pointing to it.

Flags - Note

7. URG Bit- URG bit is used to treat certain data on an urgent basis.

- When URG bit is set to 1, It indicates the receiver that certain amount of data within the current segment is urgent.
- Urgent data is pointed out by evaluating the urgent pointer field.
- The urgent data has be prioritized.
- Receiver forwards urgent data to the receiving application on a separate channel.

8. ACK Bit- ACK bit indicates whether acknowledgement number field is valid or not.

- When ACK bit is set to 1, it indicates that acknowledgement number contained in the TCP header is valid.
- For all TCP segments except request segment, ACK bit is set to 1.
- Request segment is sent for connection establishment during [Three Way Handshake](#).

9. PSH Bit- PSH bit is used to push the entire buffer immediately to the receiving application.

- When PSH bit is set to 1,
- All the segments in the buffer are immediately pushed to the receiving application.
- No wait is done for filling the entire buffer.
- This makes the entire buffer to free up immediately.

Flags - Note

NOTE

- Unlike URG bit, PSH bit does not prioritize the data.
- It just causes all the segments in the buffer to be pushed immediately to the receiving application.
- The same order is maintained in which the segments arrived.
- It is not a good practice to set PSH bit = 1.
- This is because it disrupts the working of receiver's CPU and forces it to take an action immediately.

10.RST Bit- RST bit is used to reset the TCP connection.

- When RST bit is set to 1,
- It indicates the receiver to terminate the connection immediately.
- It causes both the sides to release the connection and all its resources abnormally.
- The transfer of data ceases in both the directions.
- It may result in the loss of data that is in transit.
- This is used only when there are unrecoverable errors.
- There is no chance of terminating the TCP connection normally.

Flags - Note

11.SYN Bit-SYN bit is used to synchronize the sequence numbers.

When SYN bit is set to 1,

- It indicates the receiver that the sequence number contained in the TCP header is the initial sequence number.
- Request segment sent for connection establishment during Three way handshake contains SYN bit set to 1.

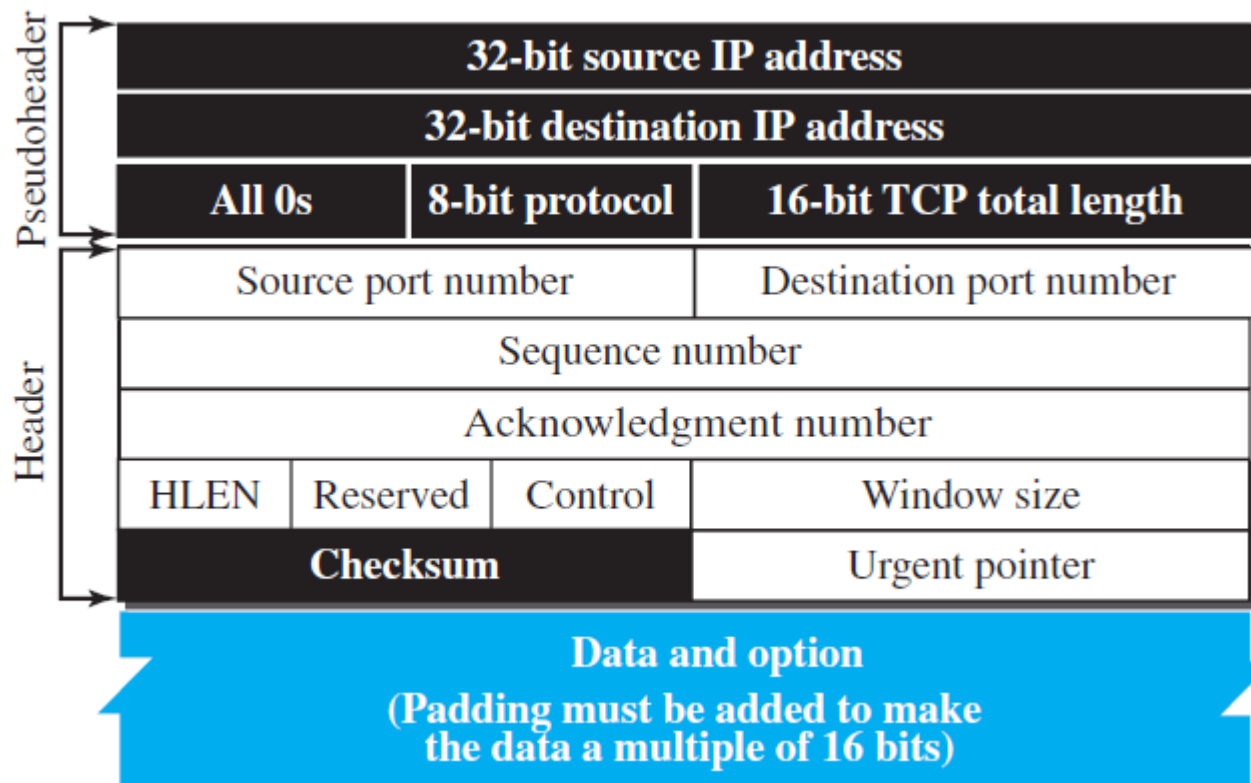
12.FIN Bit-FIN bit is used to terminate the TCP connection.

- When FIN bit is set to 1, It indicates the receiver that the sender wants to terminate the connection.
- FIN segment sent for [TCP Connection Termination](#) contains FIN bit set to 1.

The TCP Segment Header

9. Checksum-

- Checksum is a 16 bit field used for error control.
- It verifies the integrity of data in the TCP payload.
- Sender adds CRC checksum to the checksum field before sending the data.
- Receiver rejects the data that fails the CRC check.



The TCP Segment Header

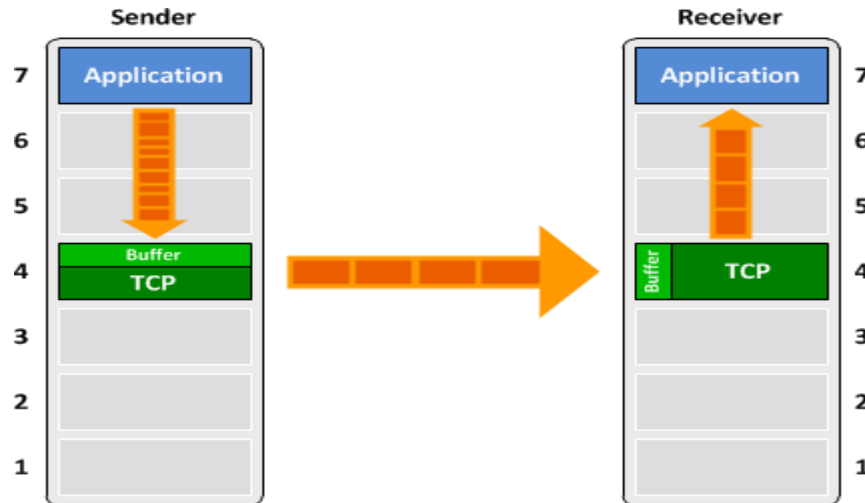
10. Urgent Pointer-

- Urgent pointer is a 16 bit field. This field is considered valid and evaluated only if the URG bit is set to 1.
- There are occasions in which an application program needs to send *urgent* bytes. The sending application program tells the sending TCP that the piece of data is urgent.
- The sender side marks some portion of the byte stream as needing **special treatment** by the application program at the receiver side
- This pointer indicates how much of the data in the segment, counting from the first byte, is urgent. (Example if the data size is 100 bytes and only first 50 bytes is urgent, the urgent pointer will have a value of 50)

11.Options field is used for several purposes.

- The size of options field vary from 0 bytes to 40 bytes. Options field is generally used for the following purposes-
- Time stamp, Window size extension, Parameter negotiation, Padding

Note- Difference between Urgent and push flags?



- The purpose of the PSH bit is to tell TCP that do not wait for the buffer to become full and send the data immediately. Similarly when the receiver receives the segment with PSH flag set, should send the data immediately to the upper layer (receiving application) without waiting for the receive buffer to become full.
- The PSH flag is also used to facilitate real-time communication via TCP

36 3.549989000 74.125.236.82 192.168.0.84 TCP 60 https > 57452 [ACK] Seq=877777514 Ack=2605484855 Win...

④ Frame 16: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0

Ethernet II, Src: AsustekC_24:1a:c1 (40:16:7e:24:1a:c1), Dst: Netgear_47:6c:06 (44:94:fc:47:6c:06)

Internet Protocol Version 4, Src: 192.168.0.84 (192.168.0.84), Dst: 74.125.236.82 (74.125.236.82)

Transmission Control Protocol, Src Port: 57452 (57452), Dst Port: https (443), Seq: 2605483508, Len: 0

Source port: 57452 (57452)

Destination port: https (443)

```
[Stream index: 1]
```

Sequence number: 2605483508

Header length: 32 bytes

Flags: 0x002 (SYN)

000, ..., = Reserved: Not set

...0 ... = Nonce: Not set

```
.... 0... .... = Congestion Window Reduced (CWR): Not set
```

```

.....,0..... = ECN-Echo: Not set

```

```

.....0..... = Urgent: Not set

```

.... 0 = Acknowledgment: Not set

```

..... 0... = Push: Not set

```

```
.....,0.. = Reset: Not set
```

```
..... 1. = Syn: Set
```

```

... .. 0 = Fin: Not set

```

window size value: 8192

```
[Calculated window size: 8192]
```

Checksum: 0x53db [validation disabled]

Source & Destination IP Addresses

Source & Destination Port Numbers

Sequence Number - 2605483508

- SYN flag set to 1

OmniSecu.com

```

0000 44 94 fc 47 6c 06 40 16 7e 24 1a c1 08 00 45 00 D..G1.@.~$....E.
0010 00 34 5e cb 40 00 80 06 a4 2c c0 a8 00 54 4a 7d .4^.@... ..TJ}
0020 ec 52 e0 6c 01 bb 9b 4c 85 f4 00 00 00 00 80 02 .R.l..L.....
0030 20 00 53 db 00 00 02 04 05 b4 01 03 03 08 01 01 .S.....
0040 04 02

```

TCP Connection Establishment

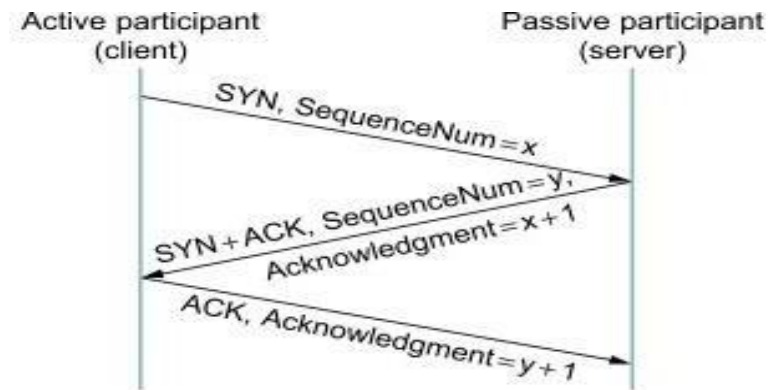
In TCP, connection-oriented transmission requires three phases:

- connection establishment,
 - data transfer, and
 - connection termination
-
- TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously
-
- The connection establishment in TCP is called ***three-way handshaking***

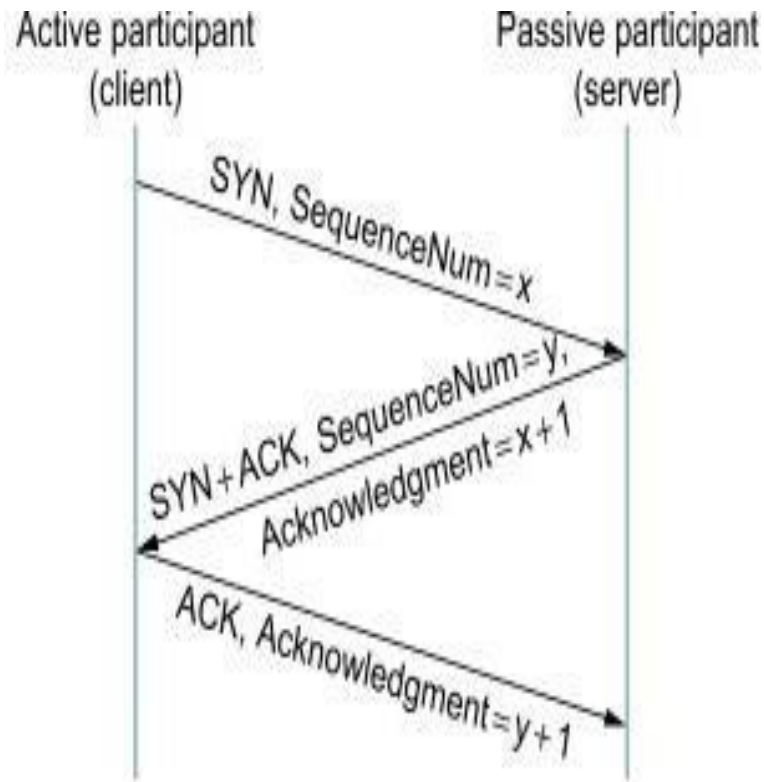
TCP Connection Establishment

TCP uses a three-way handshake to establish a reliable connection.

- The connection is full duplex, and both sides synchronize (SYN) and acknowledge (ACK) each other.
 - The exchange of these four flags is performed in three steps: SYN, SYN-ACK, ACK.
1. The client chooses an initial sequence number, set in the first SYN packet.
 2. The server also chooses its own initial sequence number, set in the SYN/ACK packet.
 3. Each side acknowledges each other's sequence number by incrementing it.
- The use of sequence and acknowledgement numbers allows both sides to detect missing or out-of-order segments.



Three Way Handshaking



*Ethernet [Wireshark 1.12.4 (v1.12.4-0-gb4861da from master-1.12)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: Expression...

No.	Time	Source	Destination	Protocol	Length	Info
8	0.47663300	10.0.2.15	198.57.208.223	TCP	66	49293->80 [SYN] Seq=0 win=65535 Len=0 MSS=1460 WS=255
9	0.53879100	198.57.208.223	10.0.2.15	TCP	60	80->49292 [SYN, ACK] Seq=0 Ack=1 win=65535 Len=0 MSS=1460
10	0.53889800	10.0.2.15	198.57.208.223	TCP	54	49292->80 [ACK] Seq=1 Ack=1 win=65535 Len=0

Frame 10: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0

Ethernet II, Src: CadmusCo_07:ca:05 (08:00:27:07:ca:05), Dst: Realtek_12:35:02 (52:54:00:12:35:02)

Internet Protocol Version 4, Src: 10.0.2.15 (10.0.2.15), Dst: 198.57.208.223 (198.57.208.223)

Transmission Control Protocol, Src Port: 49292 (49292), Dst Port: 80 (80), Seq: 1, Ack: 1, Len: 0

Source Port: 49292 (49292)

Destination Port: 80 (80)

[Stream index: 0]

[TCP segment Len: 0]

Sequence number: 1 (relative sequence number)

Acknowledgment number: 1 (relative ack number)

Header Length: 20 bytes

... 0000 0001 0000 = Flags: 0x010 (ACK)

Window size value: 65535

[Calculated window size: 65535]

[window size scaling factor: -2 (no window scaling used)]

Checksum: 0xa342 [validation disabled]

urgent pointer: 0

[SEQ/ACK analysis]

3 WAY HANDSHAKE

Frame (frame), 54 bytes

Packets: 5240 - Displayed: 5240 (100.0%) - Dropped: 0 (0.0%)

Profile: Default

(ip.addr eq 10.0.0.145 and ip.addr eq 157.166.238.17) and (tcp.port eq 54433 and tcp.port eq 80)
 Expression...
+
Bad TCP
No Broadcast Chatter
Slow HTTP
>>

No.	Time since reference or filter	Delta	Source	Destination	Protocol	Length	Info
24	5.069188000	0.000000	10.0.0.145	157.166.238.17	TCP	66	54433 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=14...
27	5.143961000	0.074773	157.166.238.17	10.0.0.145	TCP	66	80 → 54433 [SYN, ACK] Seq=0 Ack=1 Win=4380 L...
28	5.144071000	0.000110	10.0.0.145	157.166.238.17	TCP	54	54433 → 80 [ACK] Seq=1 Ack=1 Win=17520 Len=0
29	5.144274000	0.000203	10.0.0.145	157.166.238.17	HTTP	1032	GET / HTTP/1.1
30	5.222823000	0.078549	157.166.238.17	10.0.0.145	TCP	54	80 → 54433 [ACK] Seq=1 Ack=979 Win=5358 Len=0
31	5.225984000	0.003161	157.166.238.17	10.0.0.145	HTTP	481	HTTP/1.1 302 Moved Temporarily (text/html)
38	5.426077000	0.200093	10.0.0.145	157.166.238.17	TCP	54	54433 → 80 [ACK] Seq=979 Ack=428 Win=17092 L...

▶ Frame 24: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
 ▶ Ethernet II, Src: IntelCor_84:7c:5e (88:53:2e:84:7c:5e), Dst: Cisco-Li_dc:39:ed (c0:c1:c0:dc:39:ed)
 ▶ Internet Protocol Version 4, Src: 10.0.0.145, Dst: 157.166.238.17
 ▶ Transmission Control Protocol, Src Port: 54433, Dst Port: 80, Seq: 0, Len: 0

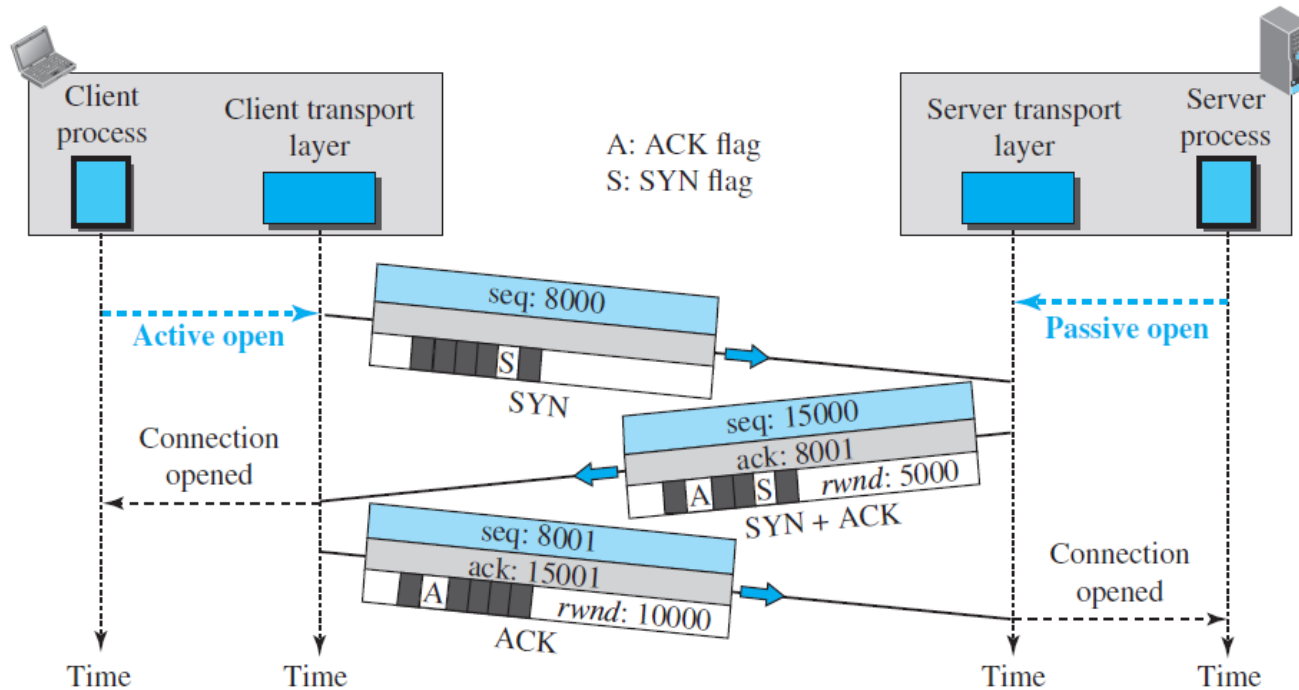
```

0000  c0 c1 c0 dc 39 ed 88 53 2e 84 7c 5e 08 00 45 00  ....9..S ..|^..E.
0010  00 34 50 b1 40 00 80 06 13 ca 0a 00 00 91 9d a6  .4P.@... ..
0020  ee 11 d4 a1 00 50 08 42 9c 51 00 00 00 00 80 02  ....P.B .Q.....
0030  20 00 3f 48 00 00 02 04 05 b4 01 03 03 02 01 01  .?H.... ..
0040  04 02
  
```

Lab 1_HTTP-CNN
 Packets: 2740 · Displayed: 7 (0.3%) · Load time: 0:0.137
Profile: TCP Plain

Three-way handshaking

Figure 24.10 Connection establishment using three-way handshaking



1. The process starts with the server. The server program tells its TCP that it is ready to accept a connection. This request is called a *passive open*.
2. The client program issues a request for an *active open*. A client that wishes to connect to an open server tells its TCP to connect to a particular server

Three-way handshaking

Client sends - SYN

- SYN flag is set
- Sequence number is called the *initial sequence number* is set
- This segment is for synchronization of sequence numbers
- A SYN segment cannot carry data, but it consumes one sequence number

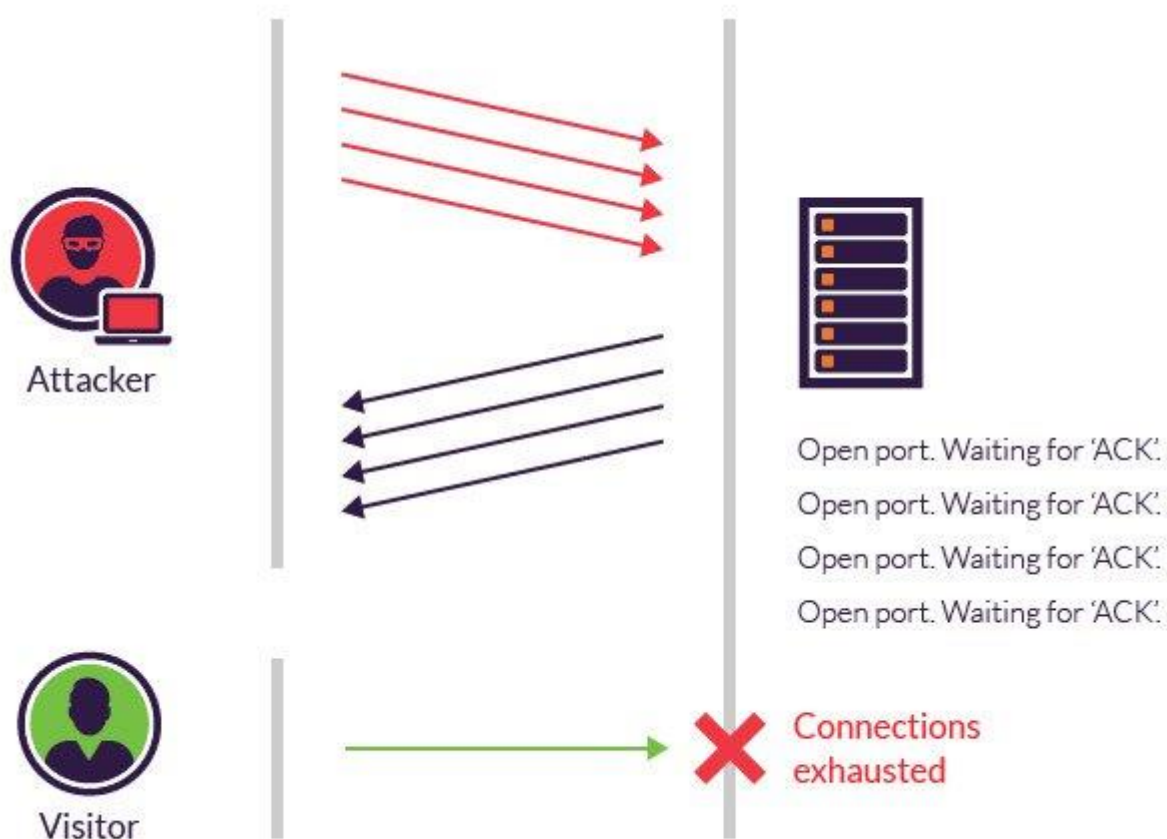
Server sends- SYN + ACK

- The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client
- The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag

Client sends – ACK

- It acknowledges the receipt of the second segment with the ACK flag

SYN Flooding Attack



This SYN flooding attack belongs to a group of security attacks known as a ***denial of service attack***. In which an attacker monopolizes a system with so many service requests that the system overloads and denies service to valid requests.

SYN Flooding Attack

- The connection establishment procedure in TCP is susceptible to a serious security problem called ***SYN flooding attack***

How?

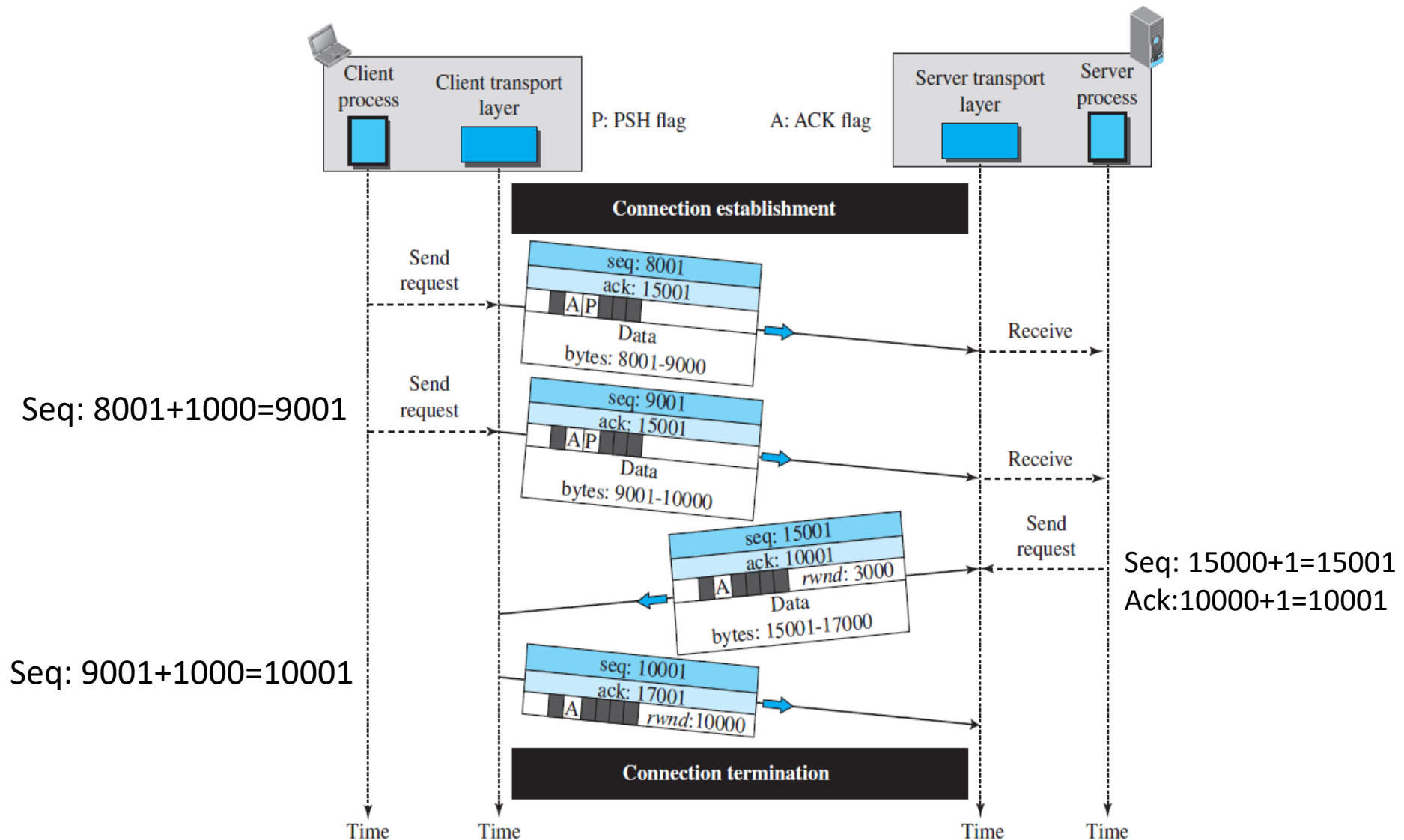
- when one or more malicious attackers send a large number of SYN segments to a server pretending that each of them is coming from a different client by faking the source IP addresses in the datagrams
- The server, assuming that the clients are issuing an active open, allocates the necessary resources, such as creating transfer control block (TCB) tables and setting timers
- TCP server then sends the SYN + ACK segments to the fake clients, which are lost

Consequences:

- however, resources are allocated without being used. The server eventually runs out of resources and may be unable to accept connection requests from valid clients
- This SYN flooding attack belongs to a group of security attacks known as a ***denial of service attack***. In which an attacker monopolizes a system with so many service requests that the system overloads and denies service to valid requests.

Data Transfer

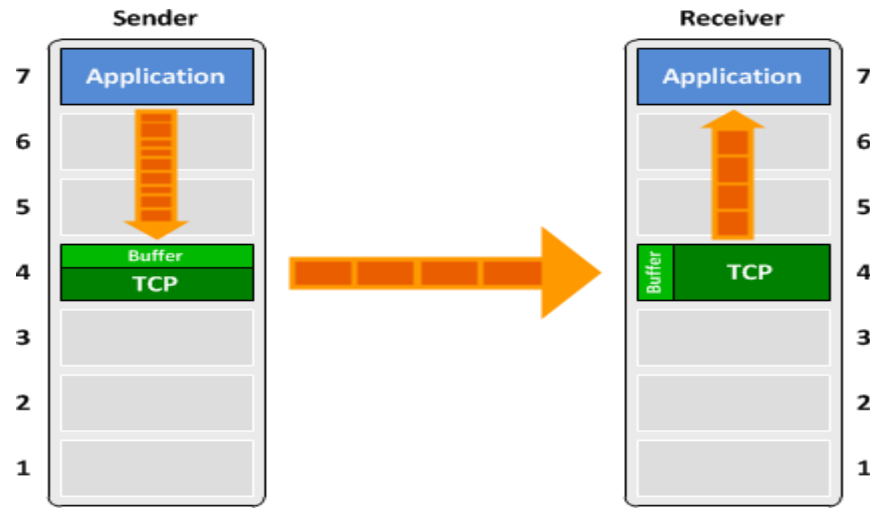
After connection is established, bidirectional data transfer can take place.



Data Transfer

- After a connection is established, the client sends 2,000 bytes of data in two segments.
- The server then sends 2,000 bytes in one segment.
- The first three segments carry both data and acknowledgment, but the last segment carries only an acknowledgment because there is no more data to be sent.
- The data segments sent by the client have the PSH (push) flag set so that the server TCP knows to deliver data to the server process as soon as they are received.
- The segment from the server, on the other hand, does not set the push flag. Most TCP implementations have the option to set or not to set this flag.

Pushing Data



Normal Scenario:

- Sender application-uses a buffer to store the stream of data. sending TCP can select the segment size
- The receiving TCP- also buffers the data when they arrive and delivers when the application program is ready or when it is convenient for the receivingTCP.

Special Occasion:

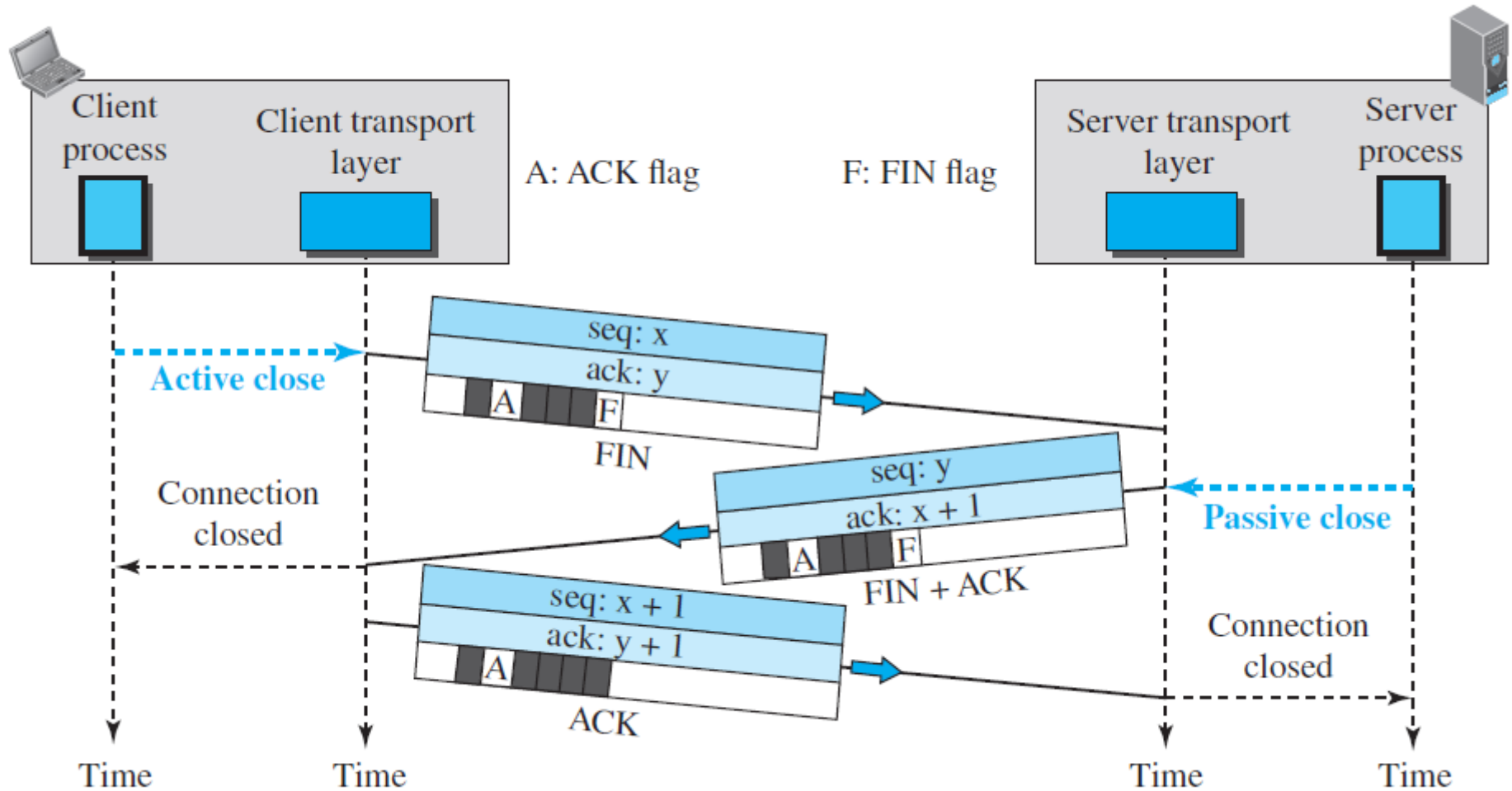
- The purpose of the PSH bit is to tell TCP that do not wait for the buffer to become full and send the data immediately. Similarly when the receiver receives the segment with PSH flag set, should send the data immediately to the upper layer (receiving application) without waiting for the receive buffer to become full.
- The PSH flag is also used to facilitate real-time communication via TCP

Urgent Data

occasions: in which an application program needs to send *urgent* bytes, some bytes that need to be treated in a special way by the application at the other end.

- **URG bit set and urgent pointer field**
- Example if the data size is 100 bytes and only first 50 bytes is urgent, the urgent pointer will have a value of 50

Connection Termination (Three-Way Handshaking)



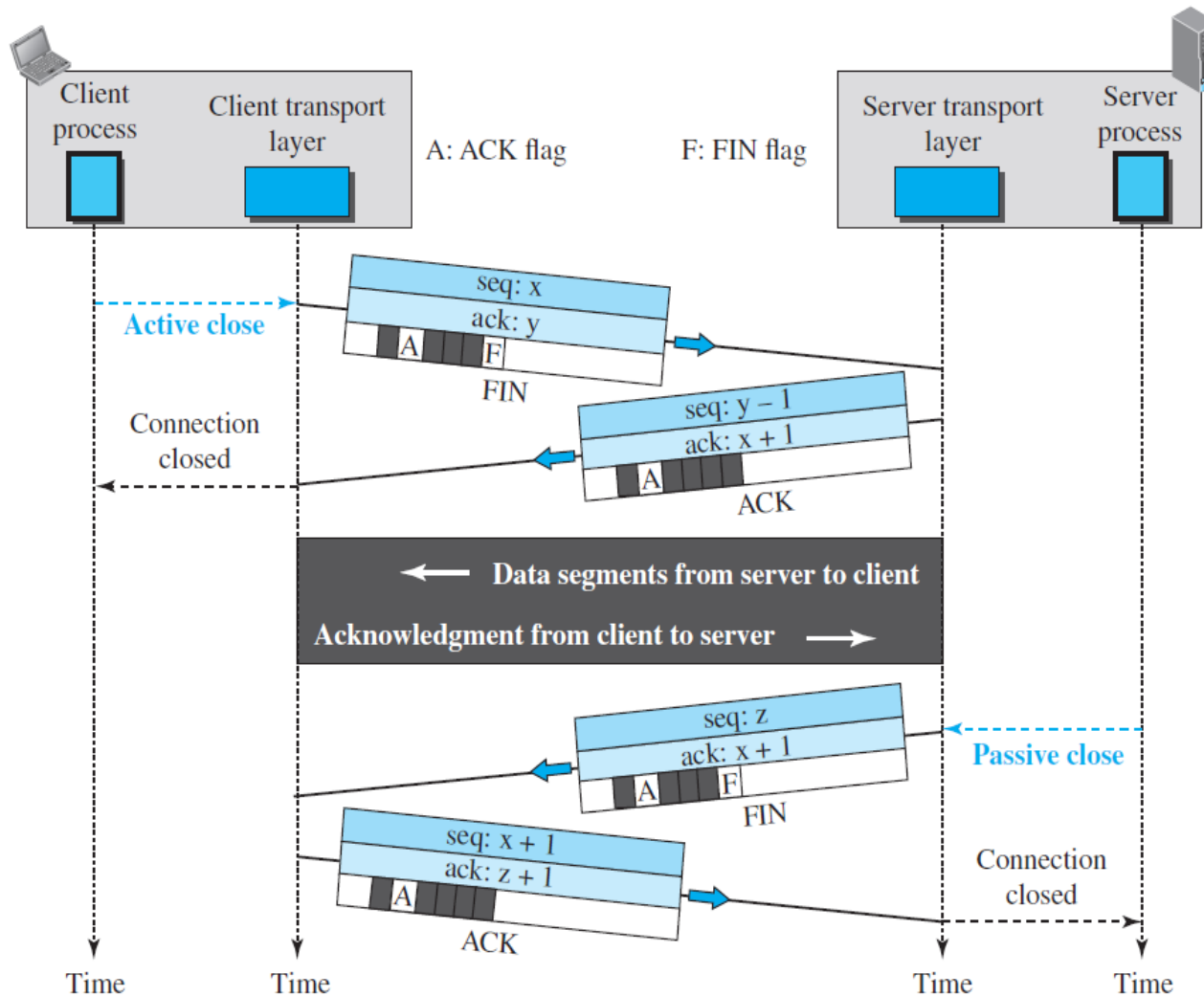
Connection Termination

(Three-Way Handshaking)

- Client: After receiving a close command from the client process, sends the first segment, a FIN segment in which the FIN flag is set
- Server: The server TCP sends FIN + ACK segment to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction
- The client TCP sends an ACK segment, to confirm the receipt of the FIN segment from the TCP server

Half-Close (four-way handshaking)

In TCP, one end can stop sending data while still receiving data. This is called a *halfclose*.



Example: sorting

State Transition Diagram

Table 24.2 *States for TCP*

<i>State</i>	<i>Description</i>
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN + ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

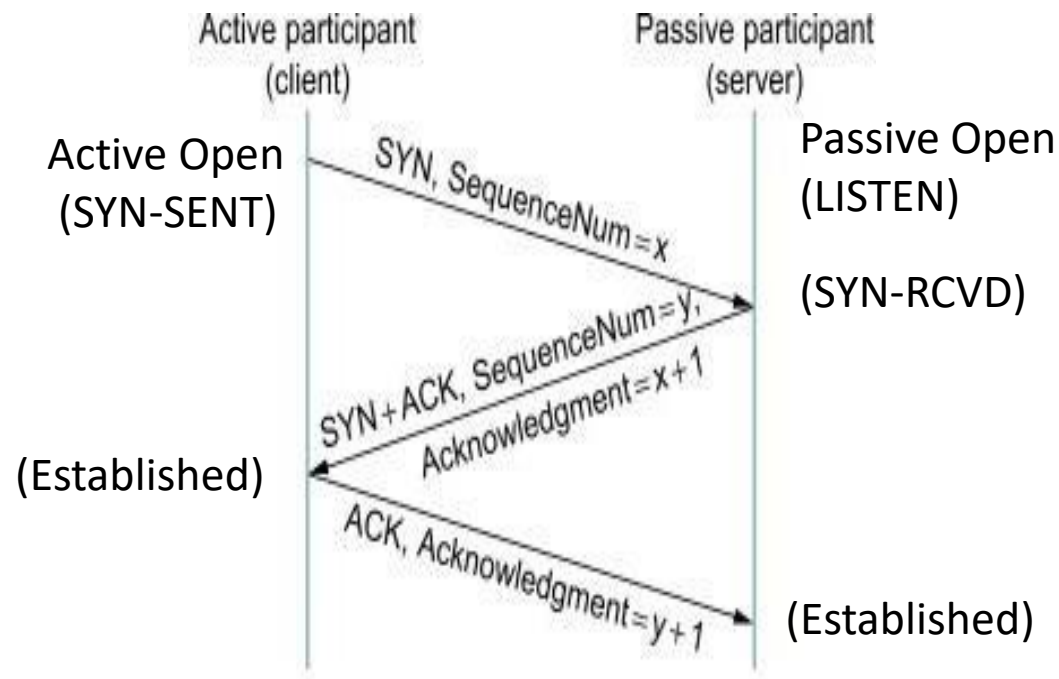
TCP Connection Management Part 1 | IIT Lecture Series

https://www.youtube.com/watch?v=wQ4_N73duO0

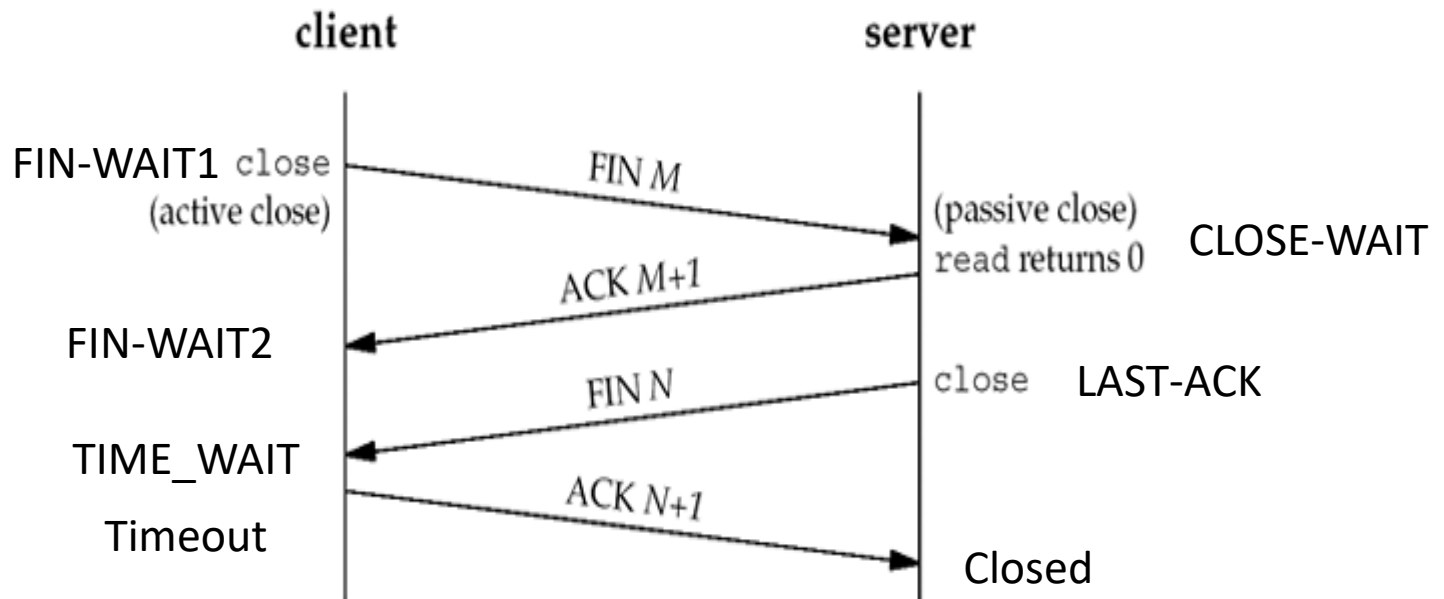
TCP Connection Management Part 2 | IIT Lecture Series

https://www.youtube.com/watch?v=IYR08ae_MOc

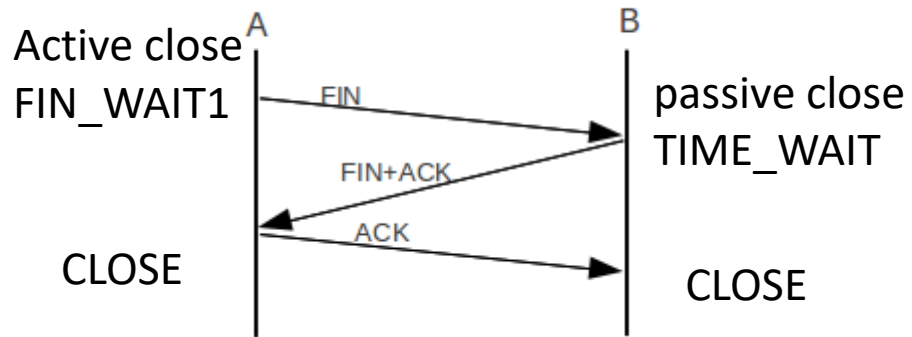
Connection Establishment



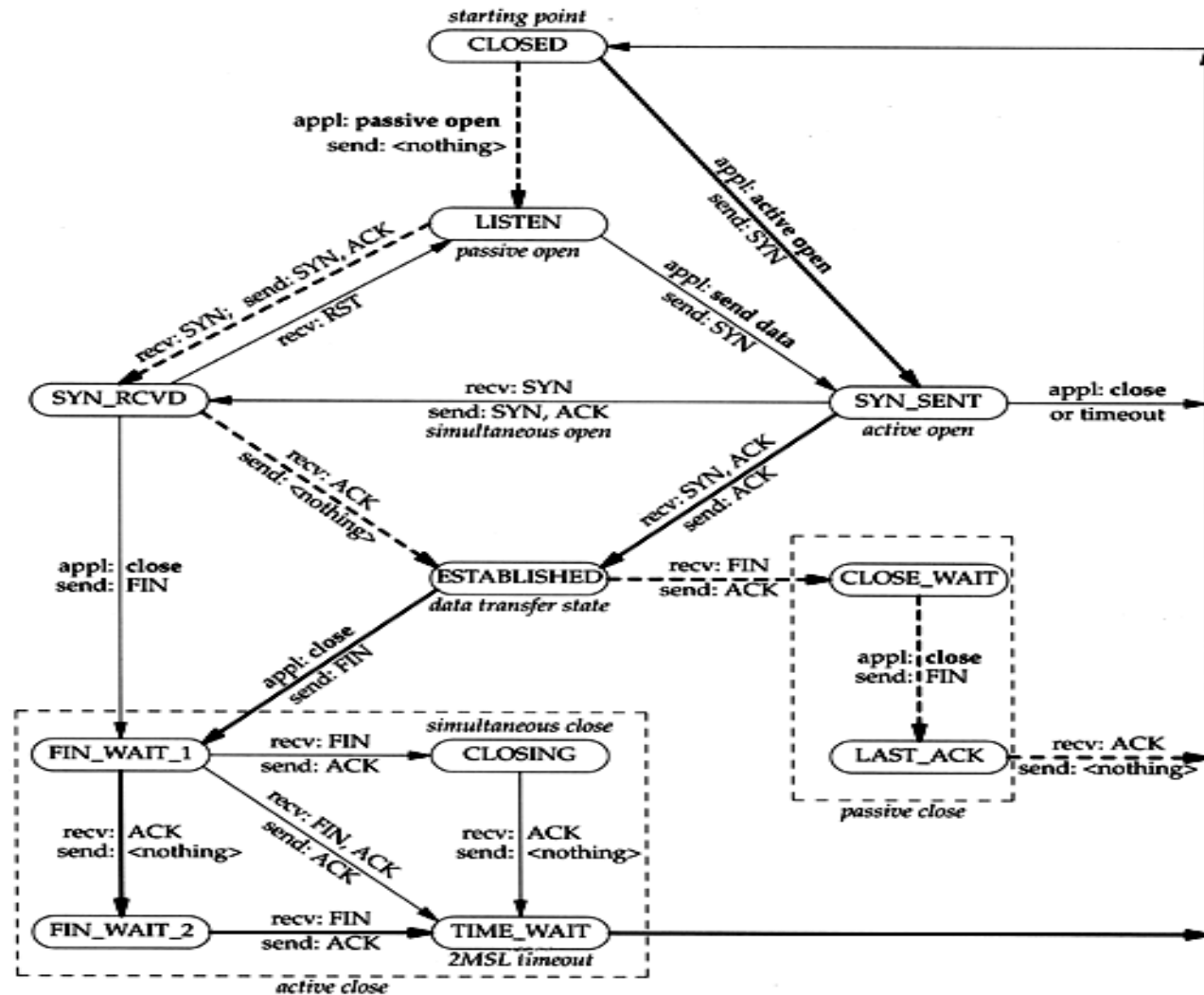
Connection Termination



Connection Termination

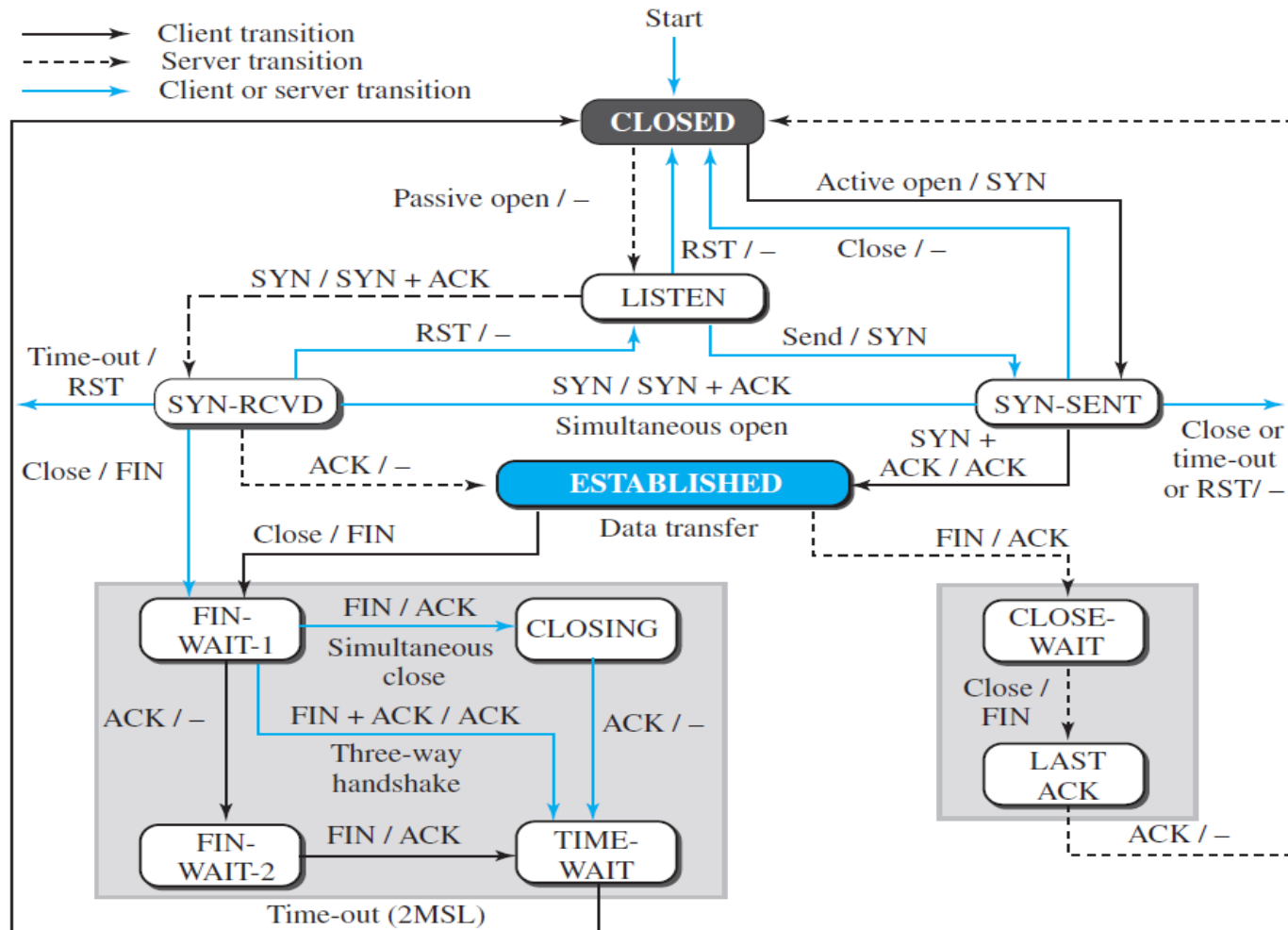


State transition diagram



—————> indicate normal transitions for client
 - - - - -> indicate normal transitions for server
 appl: —————> indicate state transitions taken when application issues operation
 recv: —————> indicate state transitions taken when segment received
 send: —————> indicate what is sent for this transition

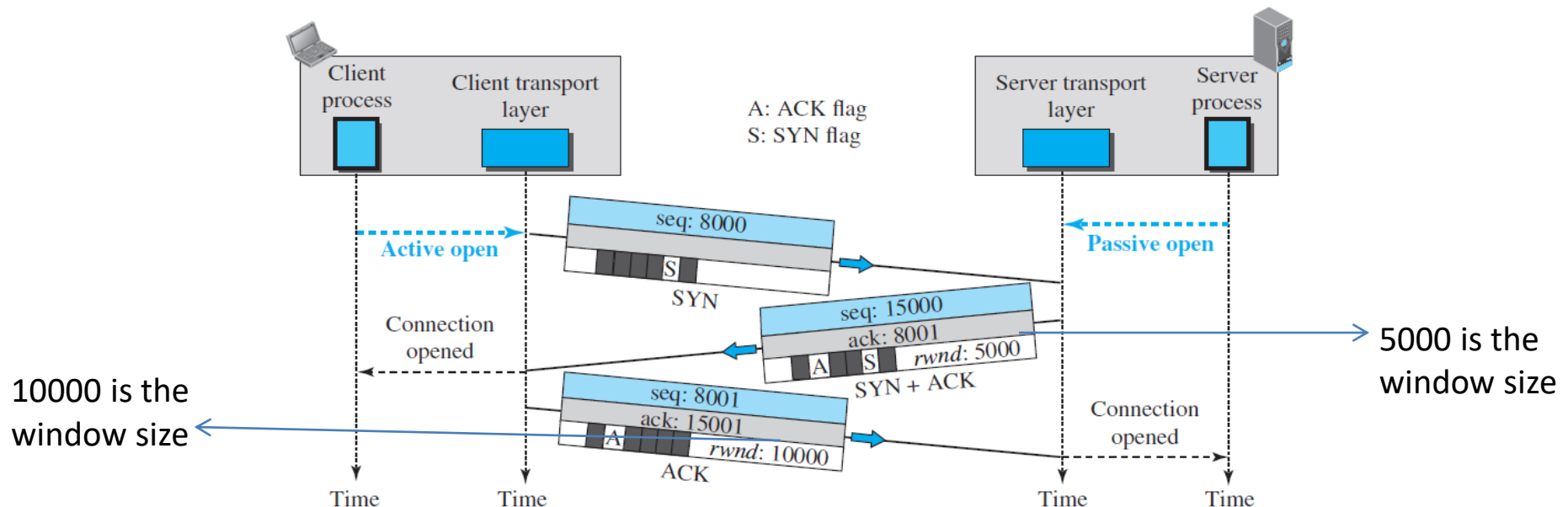
State transition diagram



Windows in TCP

- Because of overflow of data - some packet may lost (Faster sender and slower receiver –buffer) – but TCP is reliable
- TCP uses two windows (send window and receive window) for each direction of data transfer
- Sender and receiver agree on the amount of data that is going to be transferred. How many packets we are allowed to send to the receiver
- Sending window is decided by receiver during connection establishment

Figure 24.10 Connection establishment using three-way handshaking

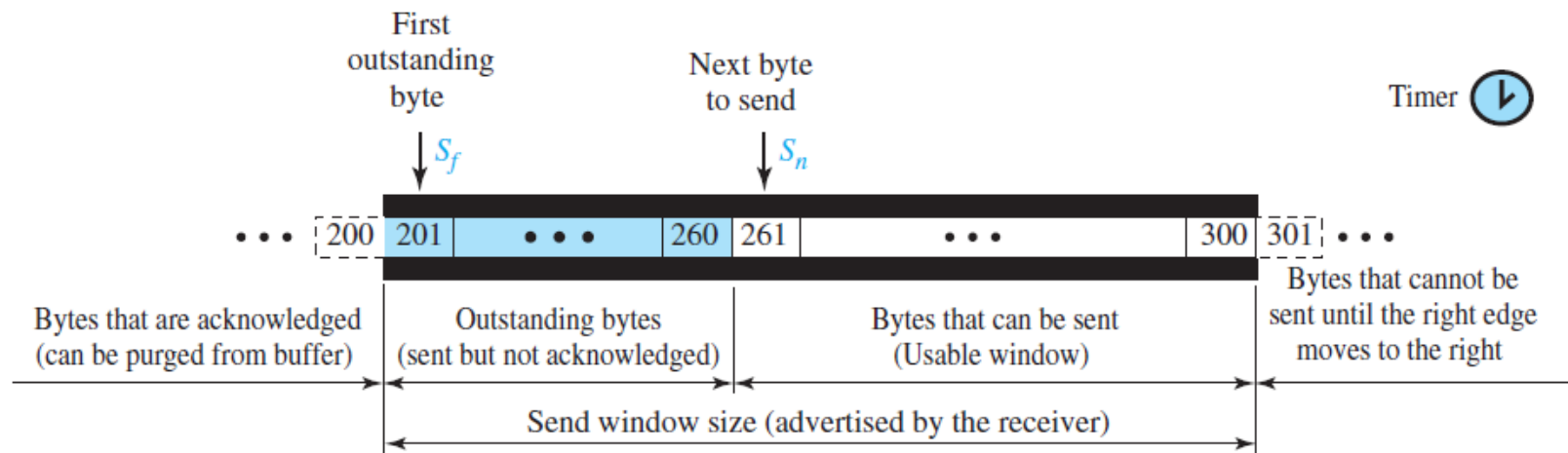


Windows in TCP

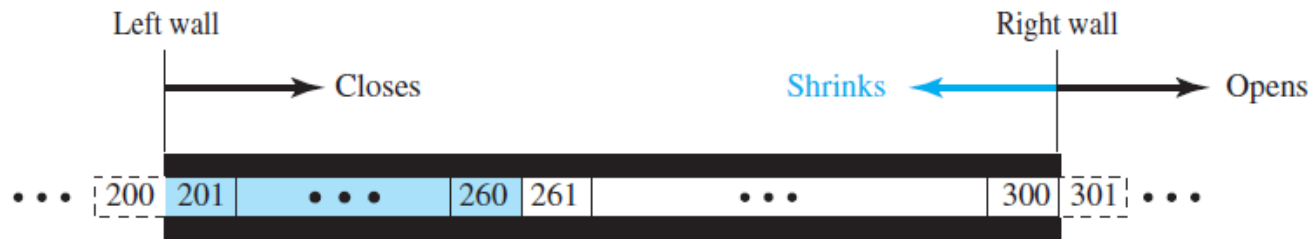
Send Window

window size is 100 bytes (201 to 300)

Figure 24.17 *Send window in TCP*



a. Send window



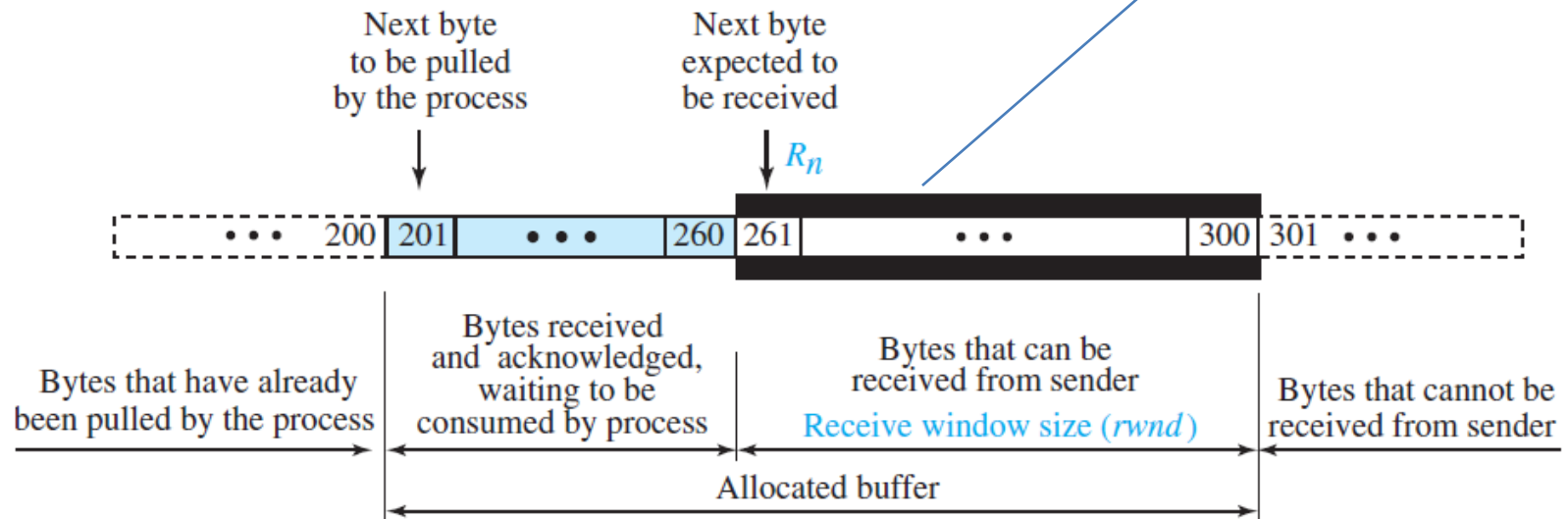
b. Opening, closing, and shrinking send window

Windows in TCP

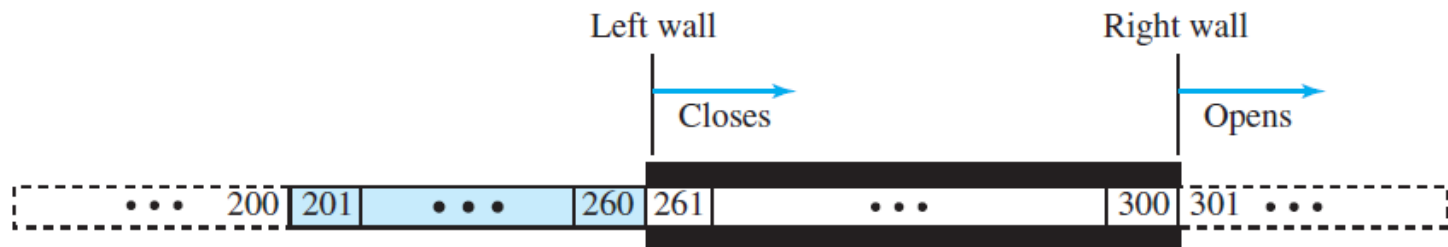
Receive Window

Receiver is capable to accommodate for 40 more bytes

Figure 24.18 Receive window in TCP



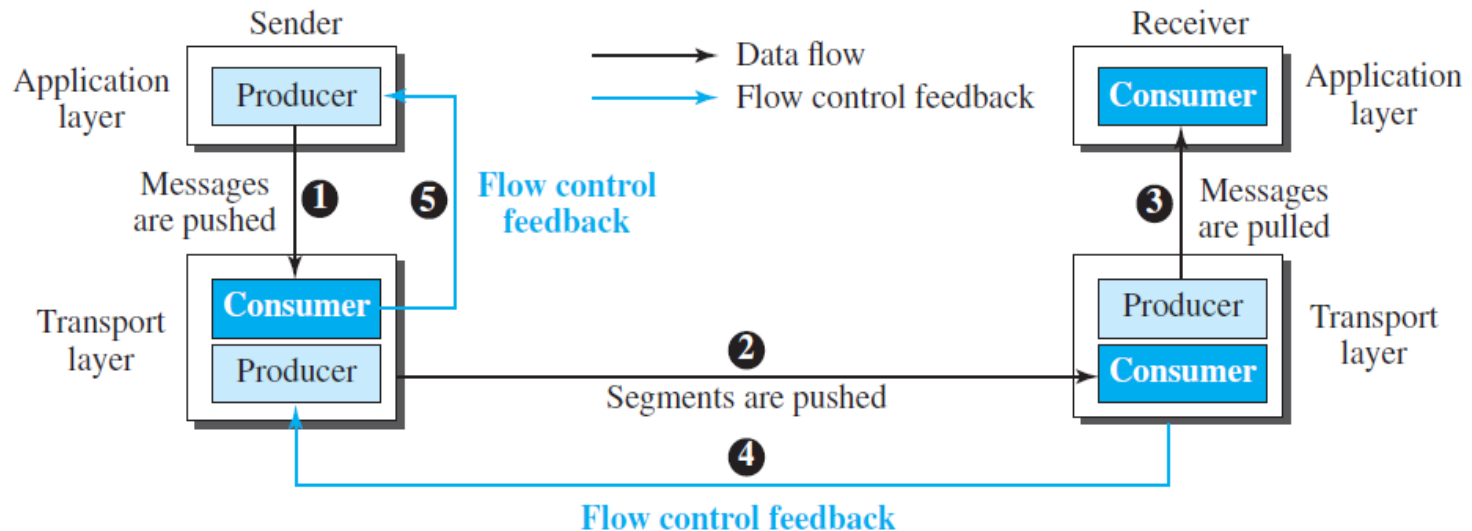
a. Receive window and allocated buffer



b. Opening and closing of receive window

Flow Control

Figure 24.19 *Data flow and flow control feedbacks in TCP*



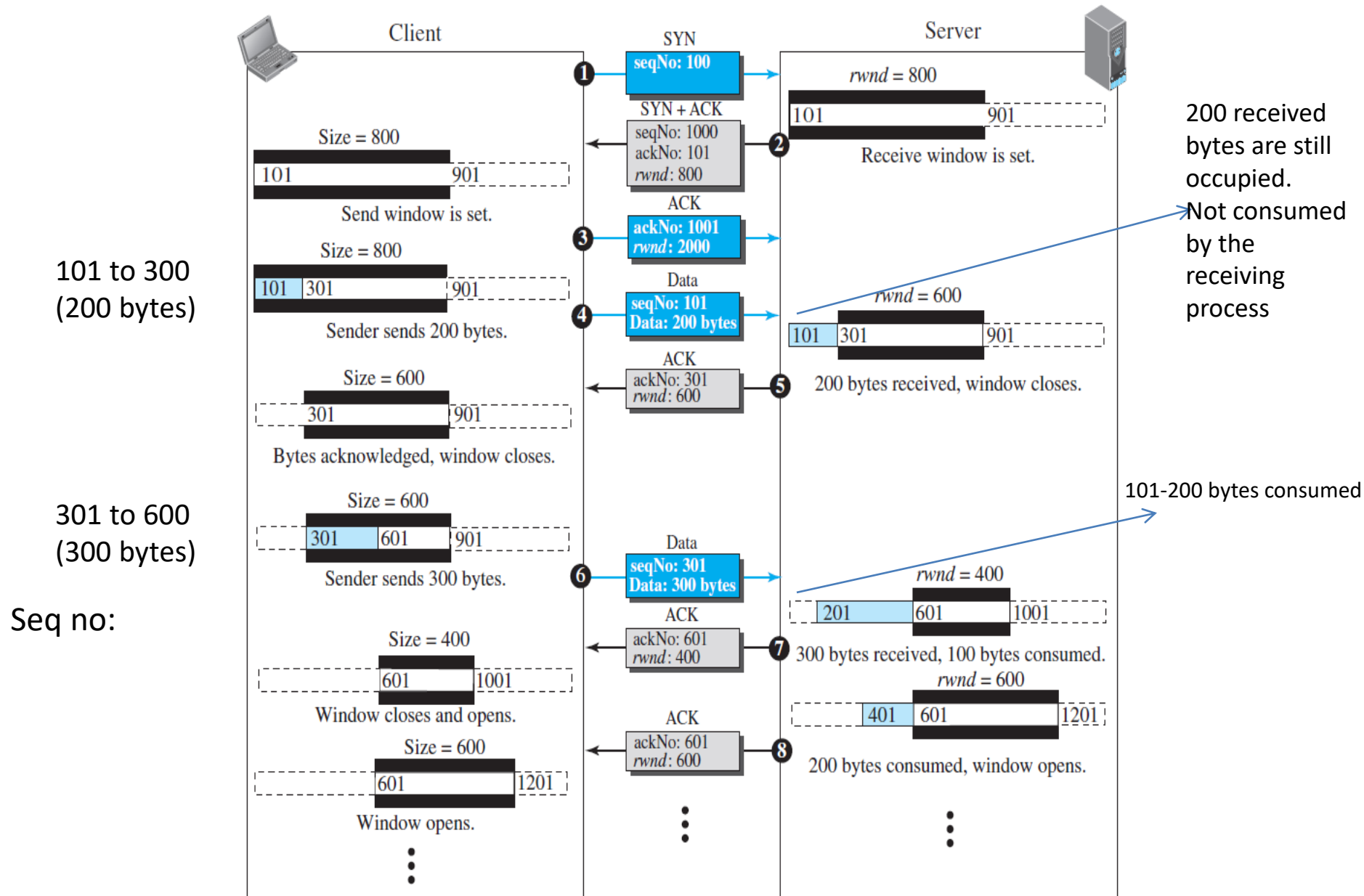
Flow control feedback: The receiving TCP controls the sending TCP; the sending TCP controls the sending process

Opening and Closing Windows

- To achieve **flow control**, TCP forces the sender and the receiver to adjust their window sizes, although the size of the buffer for both parties is fixed when the connection is established.
- The receive window closes (moves its left wall to the right) when more bytes arrive from the sender; it opens (moves its right wall to the right) when more bytes are pulled by the process. We assume that it does not shrink (the right wall does not move to the left).
- The opening, closing, and shrinking of the send window is controlled by the receiver.
- The send window closes (moves its left wall to the right) when a new acknowledgment allows it to do so.
- The send window opens (its right wall moves to the right) when the receive window size (*rwnd*) advertised by the receiver allows it to do so

(new ackNo + new *rwnd* > last ackNo + last *rwnd*).

Example of flow control (We assume only unidirectional communication from client to server. Hence only one window is shown on each side)



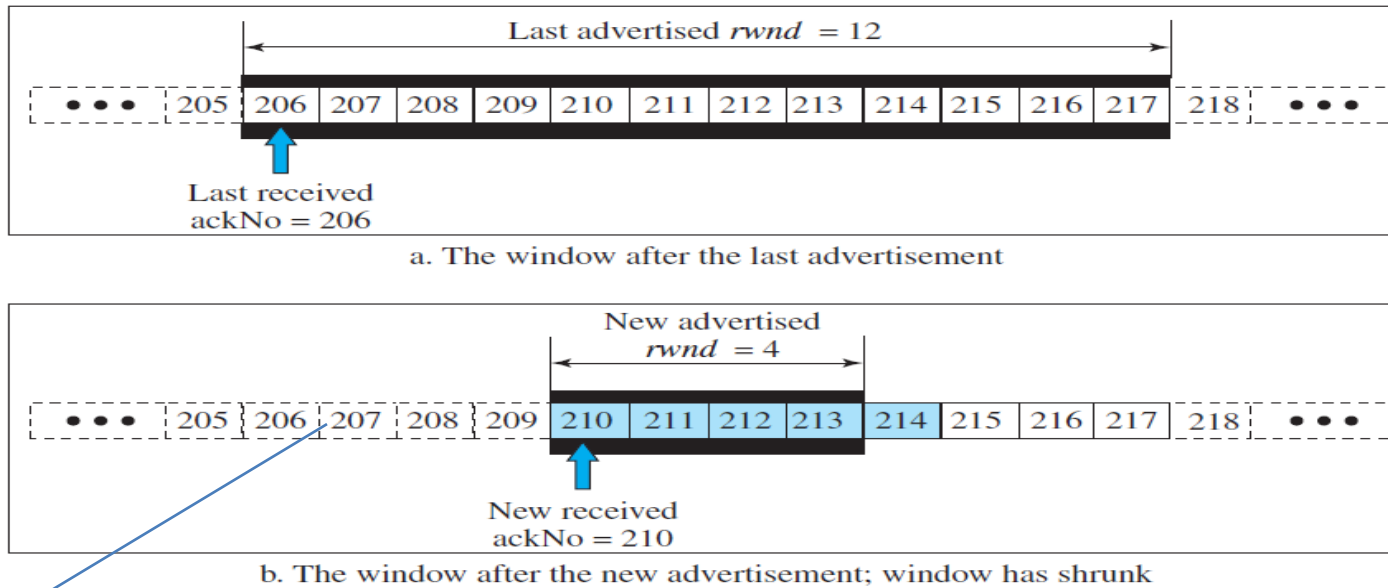
Shrinking of Windows

- The receive window cannot shrink
- The send window, on the other hand, can shrink if the receiver defines a value for *rwnd* that results in shrinking the window.
- some implementations do not allow shrinking of the send window. The limitation does not allow the right wall of the send window to move to the left.

$$\text{new ackNo} + \text{new } \textit{rwnd} \geq \text{last ackNo} + \text{last } \textit{rwnd}$$

Shrinking of Windows

Figure 24.21 Example 24.8



Receiver
Window

sender has sent bytes 206 to 214. Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which $210 + 4 < 206 + 12$.

When the send window shrinks, it creates a problem: byte 214, which has already been sent, is outside the window. It forces the relation, because the receiver does not know which of the bytes 210 to 217 has already been sent.

One way to prevent this situation is to let the receiver postpone its feedback until enough buffer locations are available in its window the receiver should wait until more bytes are consumed by its process to meet the relationship

Window Shutdown

- shrinking the send window by moving its right wall to the left is strongly discouraged
- There is one exception: the receiver can temporarily shut down the window by sending a `rwnd of 0`.
- Reason - the receiver does not want to receive any data from the sender for a while.
- In this case, the sender does not actually shrink the size of the window, but stops sending data until a new advertisement has arrived.
- even when the window is shut down by an order from the receiver, the sender can always send a segment with 1 byte of data. This is called `probing and is used to prevent a deadlock`

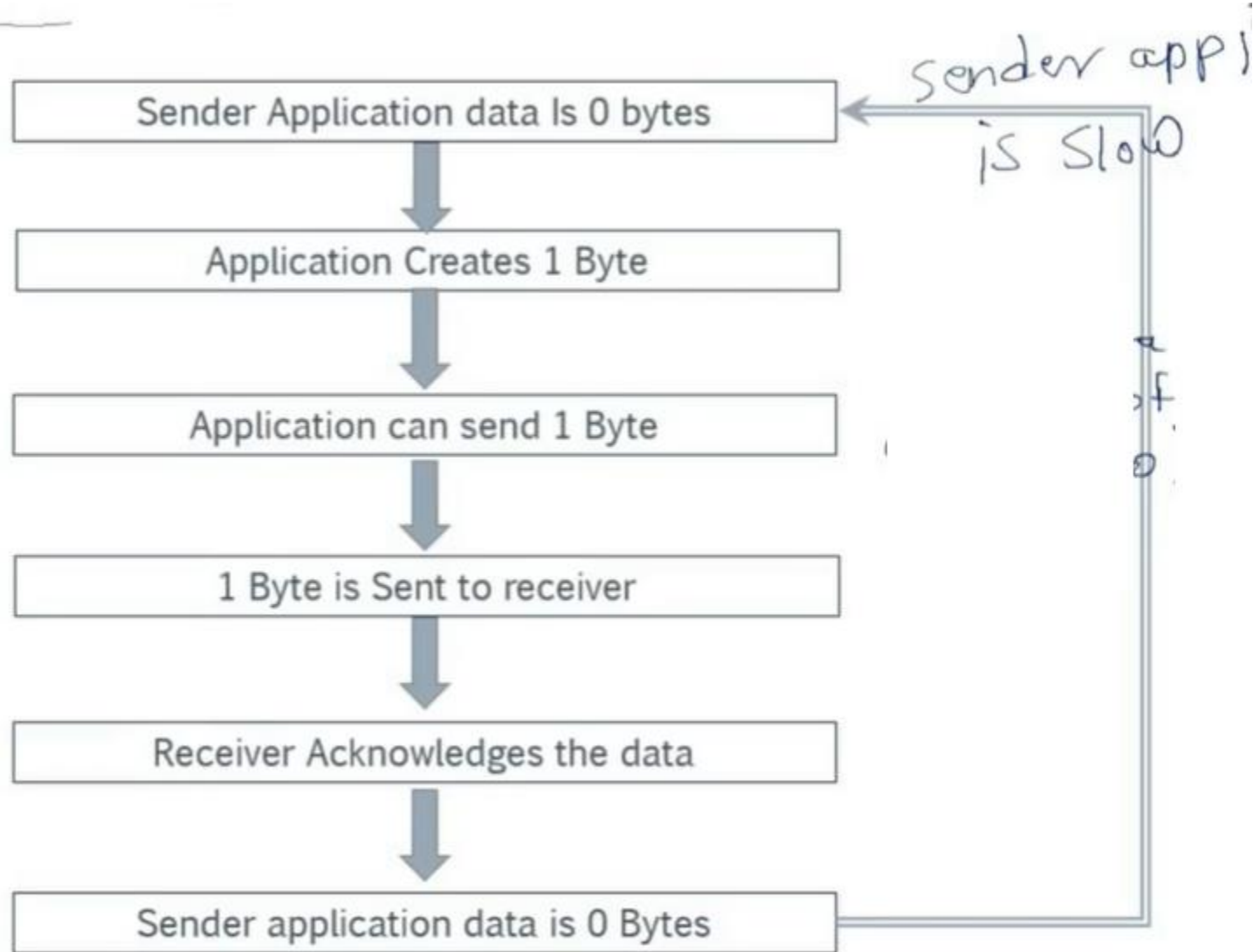
Silly Window Syndrome

- In the sliding window operation when either the sending application program **creates data slowly** or the receiving application program **consumes data slowly**, or both
- Results in the sending of data in very small segments, which **reduces the efficiency** of the operation
- Eg: if TCP sends segments containing only 1 byte of data, it means that a 41-byte datagram (20 bytes of TCP header and 20 bytes of IP header) transfers only 1 byte of user data. + the data-link layer and physical-layer overhead - inefficiency

Syndrome Created by the Sender

- The sending TCP may create a silly window syndrome if it is serving an application program that **creates data slowly** for example, 1 byte at a time
- The application program writes 1 byte at a time into the buffer of the sending TCP. If the sending TCP does not have any specific instructions, it may **create segments containing 1 byte of data**
- The solution is to prevent the sending TCP from sending the data byte by byte. The sending TCP must be forced to wait and collect data to send in a larger block. How long should the sending TCP wait? If it waits too long, it may delay the process. If it does not wait long enough, it may end up sending small segments.

Syndrome Created by the Sender



Syndrome Created by the Sender

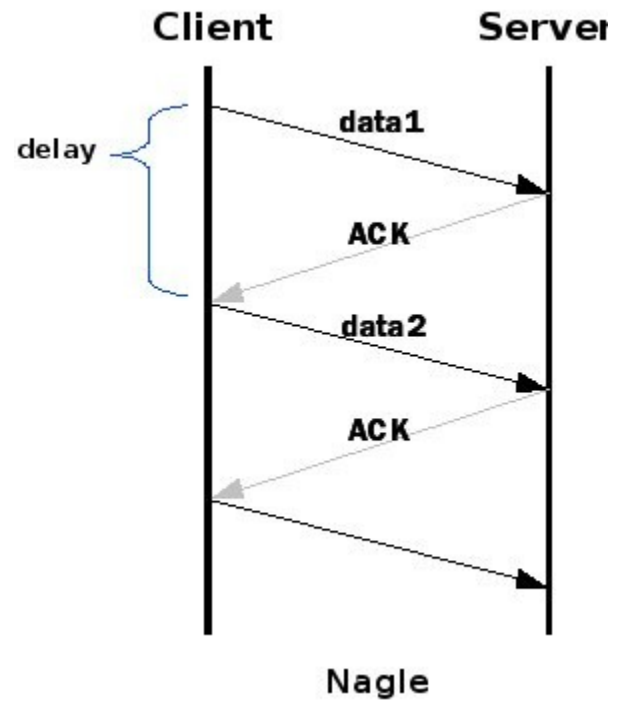
Solution: Nagle's algorithm

- 1.** The sending TCP sends the first piece of data it receives from the sending application program even if it is only 1 byte.
- 2.** After sending the first segment, the sending TCP accumulates data in the output buffer and waits until either the receiving TCP sends an acknowledgment or until enough data have accumulated to fill a maximum-size segment. At this time, the sending TCP can send the segment.
- 3.** Step 2 is repeated for the rest of the transmission. Segment 3 is sent immediately if an acknowledgment is received for segment 2, or if enough data have accumulated to fill a maximum-size segment.

Features:

- Simplicity
- If the application program is faster than the network, the segments are larger (maximum-size segments). If the application program is slower than the network, the segments are smaller (less than the maximum segment size).

waits until either the receiving TCP sends
an acknowledgment **or**
enough data have accumulated to fill a
maximum-size segment



Syndrome Created by the Receiver

- The receiving TCP may create a silly window syndrome if it is serving an application program that **consumes data slowly**, for example, 1 byte at a time.
- Eg: input buffer of the receiving TCP is 4 kilobytes. The sender sends the first 4 kilobytes of data. The receiver stores it in its buffer and buffer is full receiving application reads the first byte of data and advertises and advertise a window size of 1 byte

Silly Window Syndrome (Receiver)



Solution: Syndrome Created by the Receiver

Solution for Silly Window Syndrome at receiver side

Clarke's Algorithm

1. Receiver acknowledges a data immediately, but announces a window size as zero.
2. Receiver should wait until it has a decent amount of space available.
3. Receiver should then advertise that window size to the sender.

Delayed Acknowledgement

In this solution segment is not acknowledged immediately instead receiver waits until sufficient amount space is created in incoming buffer.

Syndrome Created by the Receiver

- Delayed acknowledgment also has another advantage: it reduces traffic.
- The receiver does not have to acknowledge each segment. However, there also is a disadvantage in that the delayed acknowledgment may result in the sender unnecessarily retransmitting the unacknowledged segments.

Question

- With an example, explain flow control in TCP.
- Define silly window syndrome? What are the solutions to avoid it (Nagles and Clarkes solution)?

Error Control

- TCP is a reliable transport-layer protocol - deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.
- TCP provides reliability using error control. Error control includes mechanisms for detecting and resending corrupted segments, resending lost segments, storing out-of order segments until missing segments arrive, and detecting and discarding duplicated segments.
- Error control in TCP is achieved through the use of three simple tools: **checksum, acknowledgment, and time-out.**

Error Control

Checksum

- TCP uses a 16-bit checksum that is mandatory in every segment.
- checksum field - used to check for a corrupted segment

Acknowledgment

TCP uses acknowledgments to confirm the receipt of data segments

Acknowledgment Type: TCP used only one type of acknowledgment: cumulative acknowledgment, selective acknowledgment

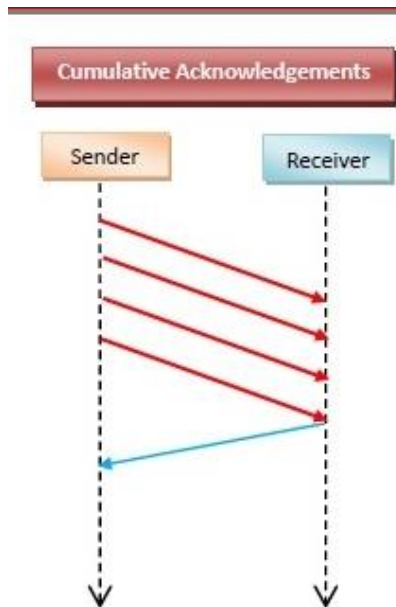
Cumulative Acknowledgment (ACK):

- The receiver advertises the next byte it expects to receive
- Cumulative acknowledgement is a process in which the receiver sends a single acknowledgement in response to a finite number of frames received.
- Sometimes referred to as *positive cumulative acknowledgment*, or ACK
- The word *positive* indicates that no feedback is provided for discarded, lost, or duplicate segments.
- The 32-bit ACK field in the TCP header is used for cumulative acknowledgments, and its value is valid only when the ACK flag bit is set to 1.

Error Control

Selective Acknowledgment (SACK)

- A SACK does not replace an ACK, but reports additional information to the sender.
- A SACK reports a block of bytes that is out of order, and also a block of bytes that is duplicated, i.e., received more than once.
- SACK is implemented as an option at the end of the TCP header.



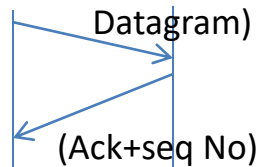
Generating Acknowledgments

(When does a receiver generate acknowledgments?)

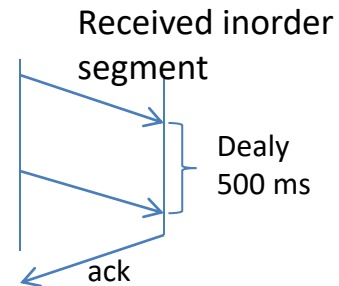
The most common rules here:

1. When end A sends a data segment to end B, it must include (piggyback) an acknowledgment that gives the next sequence number it expects to receive.

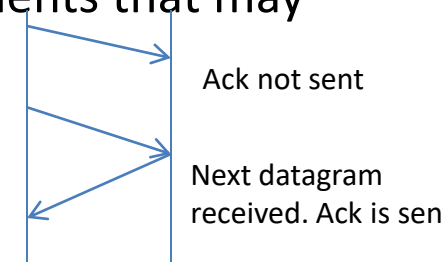
This rule decreases the number of segments needed and therefore reduces traffic



2. The receiver needs to delay sending an ACK segment if there is only one outstanding in-order segment. This rule reduces ACK segments.

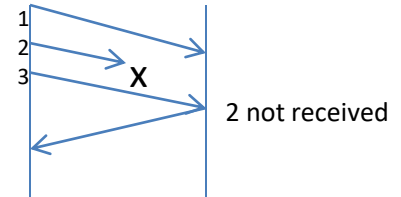


3. There should not be more than two in-order unacknowledged segments at any time. This prevents the unnecessary retransmission of segments that may create congestion in the network.



The most common rules here:

4. When a segment arrives with an out-of-order sequence number that is higher than expected, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. This leads to the *fast retransmission* of missing segments



5. When a missing segment arrives, the receiver sends an ACK segment to announce the next sequence number expected. This informs the receiver that segments reported missing have been received.

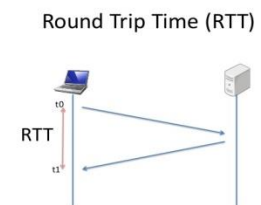
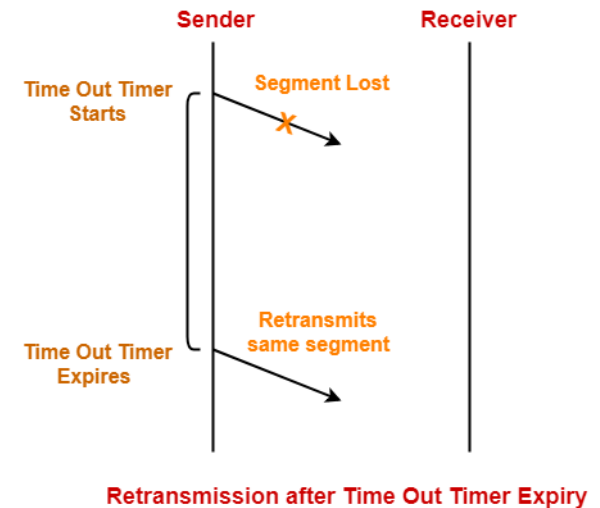
6. If a duplicate segment arrives, the receiver discards the segment, but immediately sends an acknowledgment indicating the next in-order segment expected. This solves some problems when an ACK segment itself is lost.

Retransmission

- When sender discovers that the segment sent by it is lost, it retransmits the same segment to the receiver.
- Sender discovers that the TCP segment is lost when-
 - Either Time Out Timer expires
 - Or it receives three duplicate acknowledgements

Retransmission time-out (RTO)

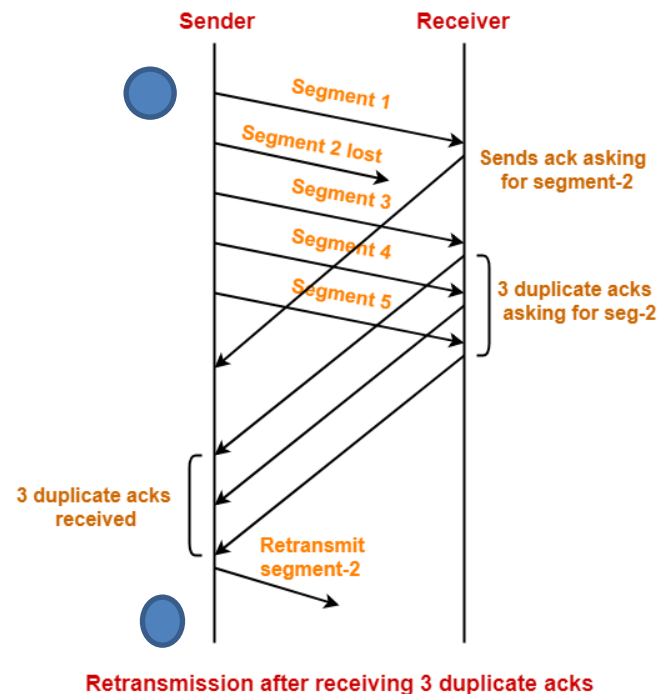
- The sending TCP maintains one **retransmission time-out (RTO)** for each connection. When the timer matures, i.e. times out, TCP resends the segment in the front of the queue and restarts the timer.
- The value of RTO is dynamic in TCP and is updated based on the **round-trip time (RTT)** of segments
- RTT is the time needed for a segment to reach a destination and for an acknowledgment to be received.



Retransmission

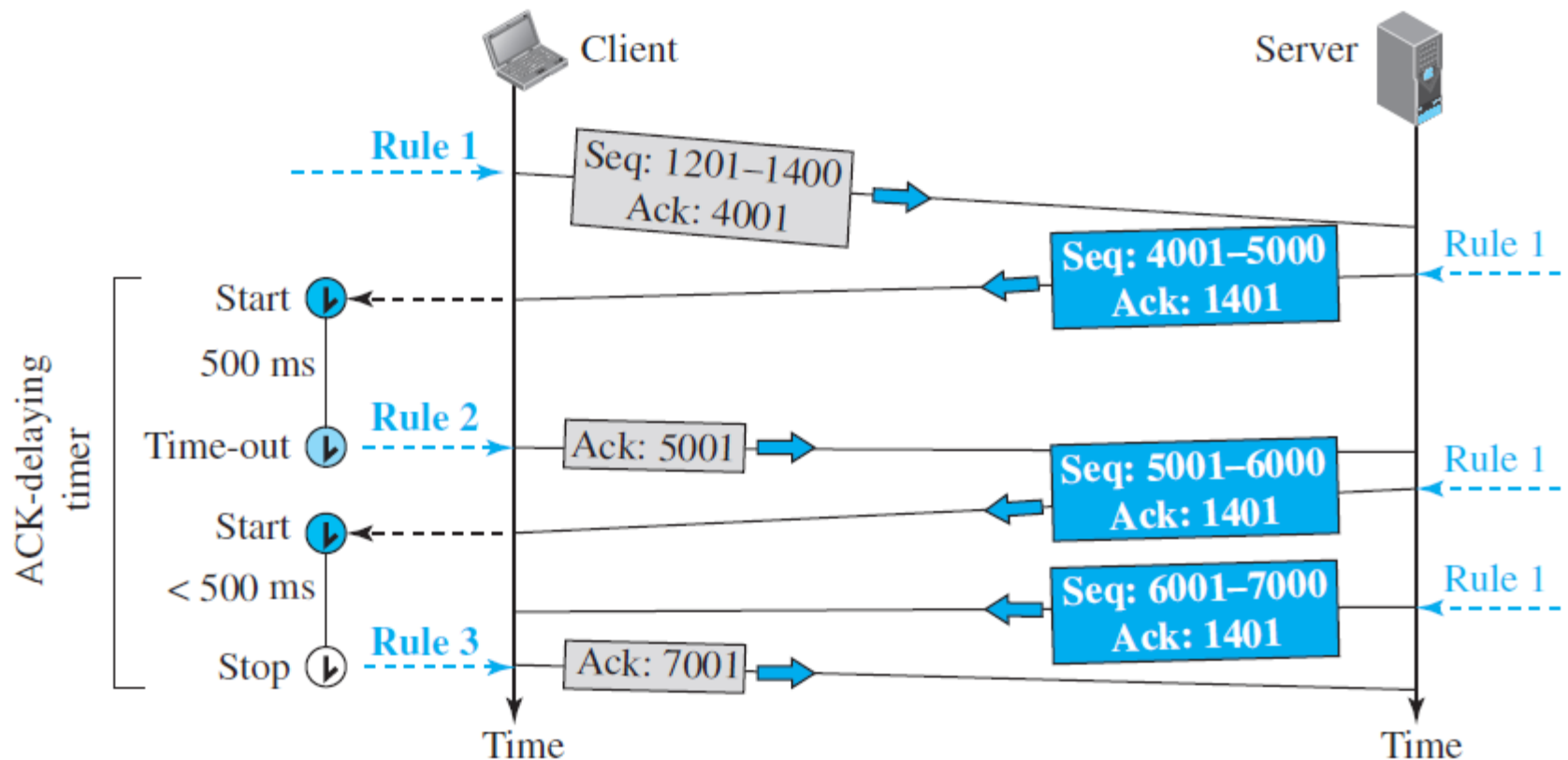
Retransmission after Three Duplicate ACK Segments

- *Fast retransmission*
- If three duplicate acknowledgments (i.e., an original ACK plus three exactly identical copies) arrive for a segment, the next segment is retransmitted without waiting for the time-out.



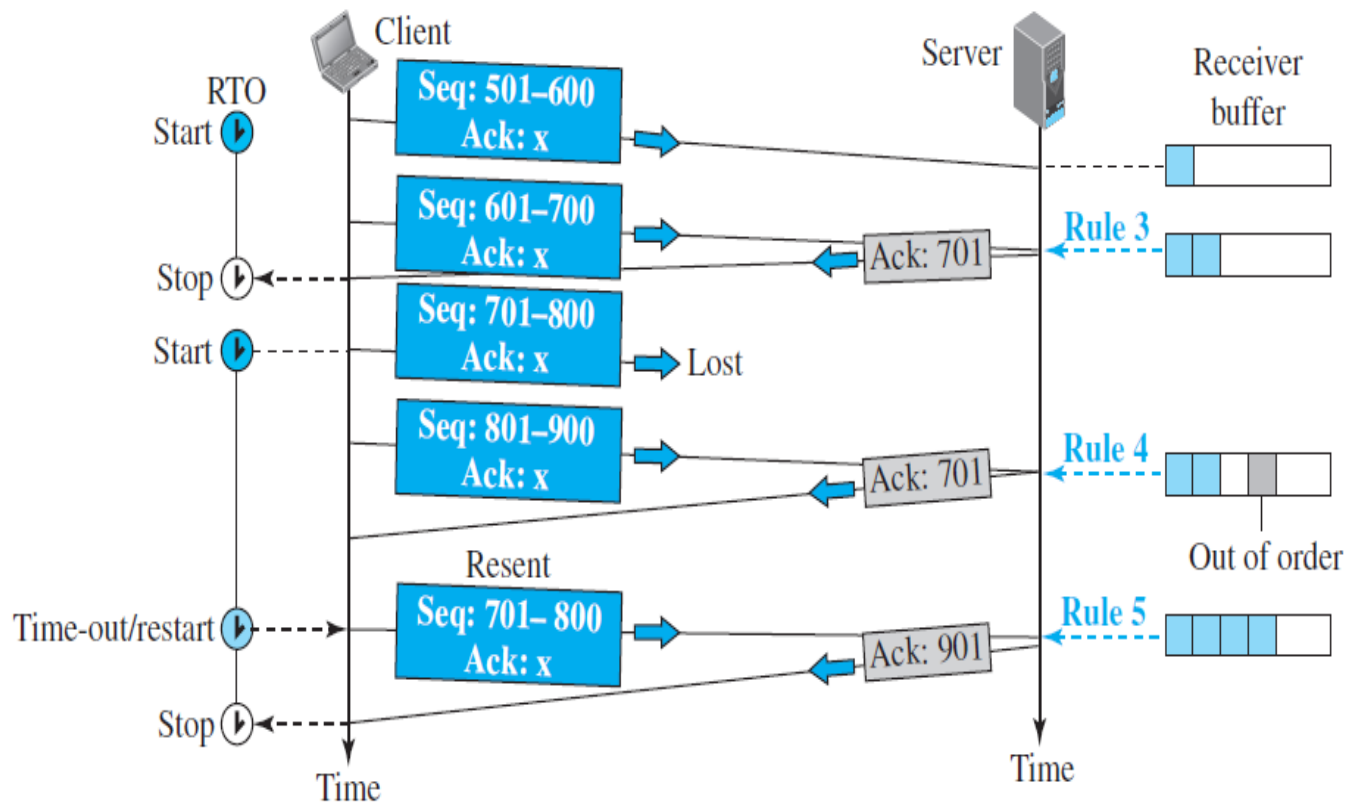
Some Scenarios (error control issues)

Figure 24.24 *Normal operation*



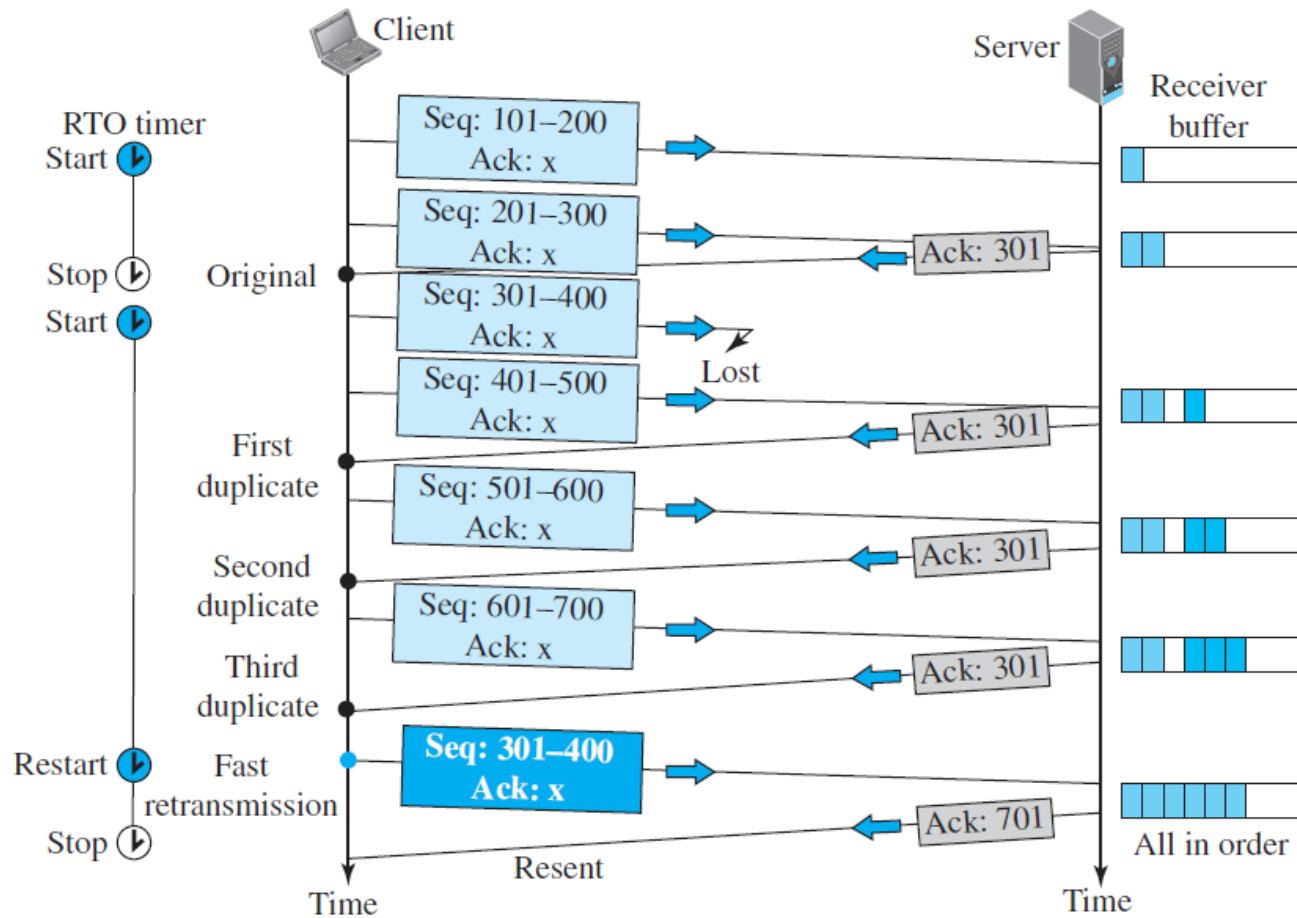
Some Scenarios (Lost segment)

Figure 24.25 *Lost segment*



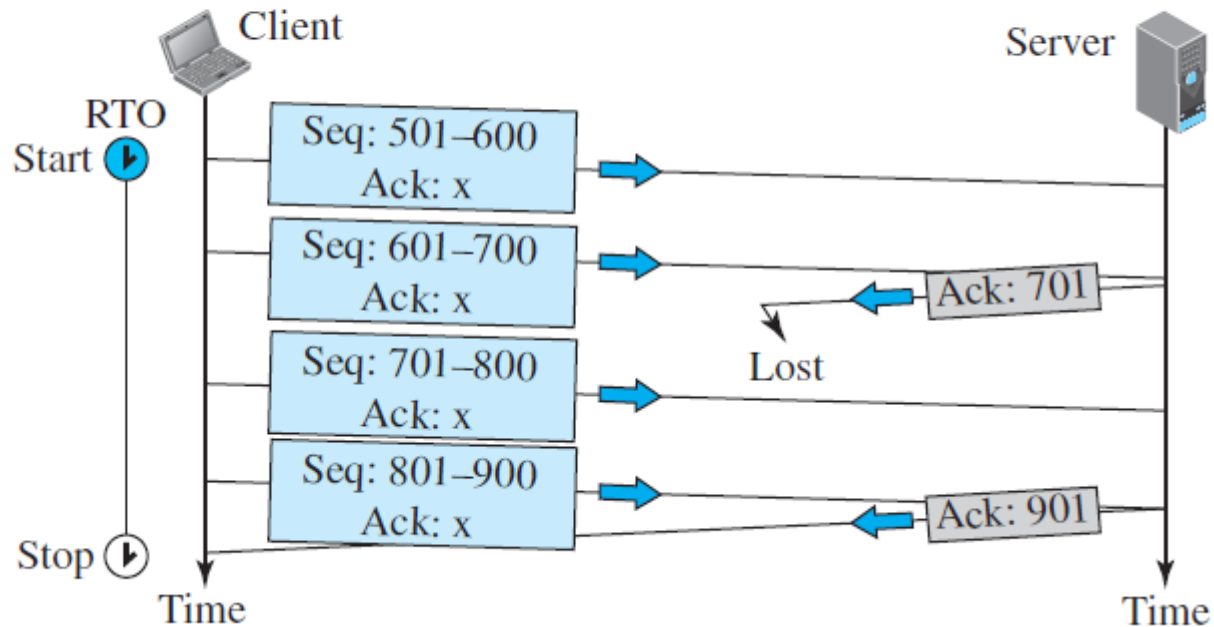
Some Scenarios (Fast Retransmission)

Figure 24.26 Fast retransmission



Some Scenarios (Lost acknowledgment)

Figure 24.27 *Lost acknowledgment*

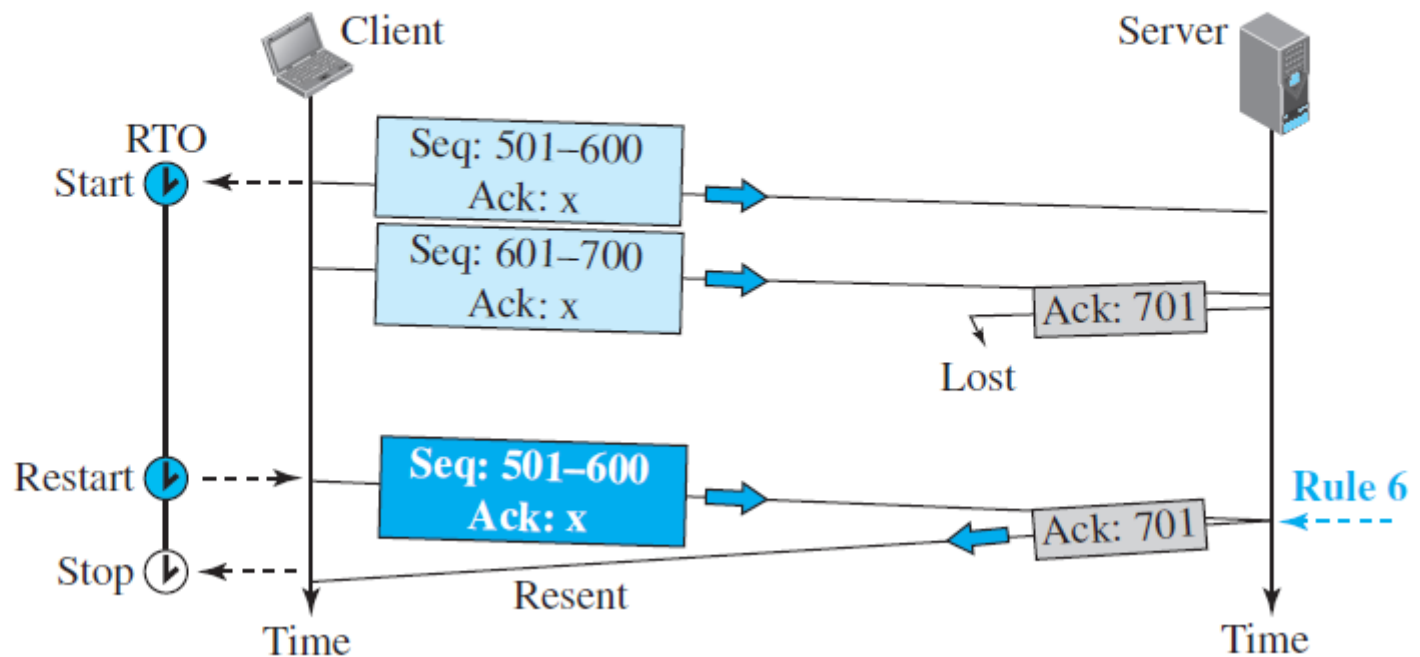


Packets have reached safely

Some Scenarios

(Lost Acknowledgment Corrected by Resending a Segment)

Figure 24.28 *Lost acknowledgment corrected by resending a segment*



Congestion Detection

Using the occurrence of two events:

- 1- time out (RTO) with no ACK.
- 2- receiving three duplicates ACKs

Congestion Policies algorithms

- 1- Slow Start
- 2- Congestion Avoidance
- 3- Fast Recovery

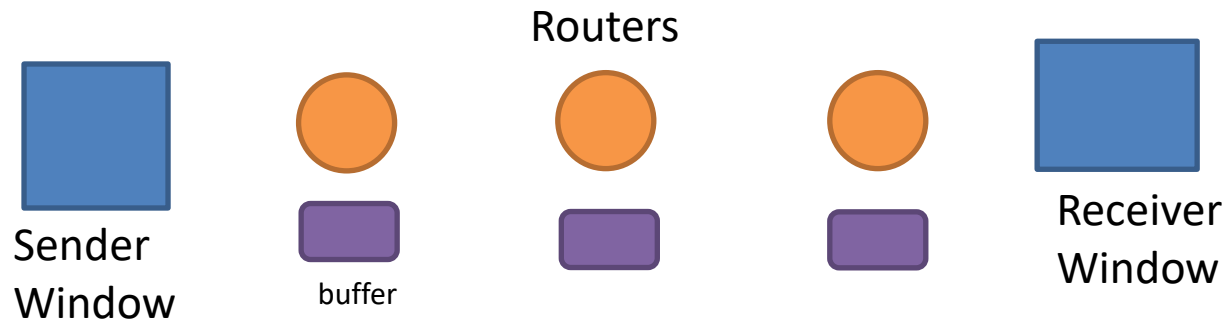
TCP versions

- 1- Tahoe TCP (treated both events similarly)
- 2- Reno TCP (treated both events differently)

Congestion

- Load of the network > capacity of the network
- Fast sender and slower receiver
- Traffic overloading – packet loss
- **Congestion** causes packets to be dropped on the network due to buffer overflow

Congestion Window



- The send window is controlled by the receiver using the value of ***rwnd***. This strategy guarantees that the receive window is never overflowed with the received bytes (no end congestion)
- TCP is an end-to-end protocol that uses the service of IP. The congestion in the router is in the IP territory and should be taken care of by IP. IP is a simple protocol with no congestion control. TCP, itself, needs to be responsible for this problem.

Congestion Window

- To control the number of segments to transmit, TCP uses another variable called a ***congestion window, cwnd***, whose size is controlled by the congestion situation in the network.
- The *cwnd* variable and the *rwnd* variable together define the size of the send window in TCP

cwnd variable - congestion in the middle (network)

rwnd variable - related to the congestion at the end

- The actual size of the window is the minimum of these two.

Actual window size = minimum (*rwnd*, *cwnd*)

Congestion Detection

- The TCP sender uses the occurrence of two events as signs of congestion in the network: **time-out and receiving three duplicate ACKs**
- **time-out:** If a TCP sender does not receive an ACK for a segment or a group of segments before the time-out occurs, it assumes that the corresponding segment or segments are lost and the loss is due to congestion
- the receiving of **three duplicate ACKs**

Versions of TCP: *Tahoe TCP and Reno TCP*

Congestion Policies

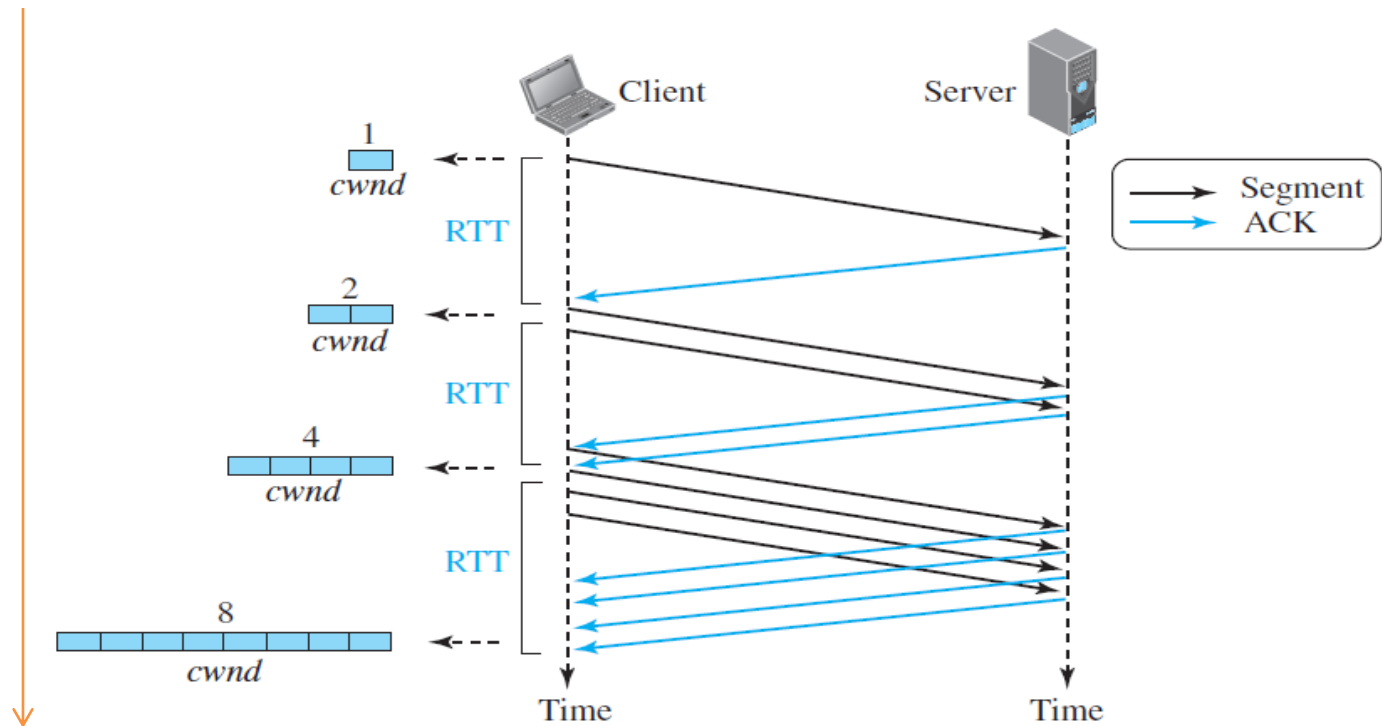
TCP's general policy for handling congestion is based on three algorithms:

- **slow start –exponential increase**
- **congestion avoidance- additive increase**
- **fast recovery.**

Figure 24.29 *Slow start, exponential increase*

**Slow start
& Exponential
increase**

Exponential
Growth



If an ACK arrives, $cwnd = cwnd + 1$

Start

After 1 RTT

After 2 RTT

After 3 RTT

→ $cwnd = 1 \rightarrow 2^0$

→ $cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$

→ $cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$

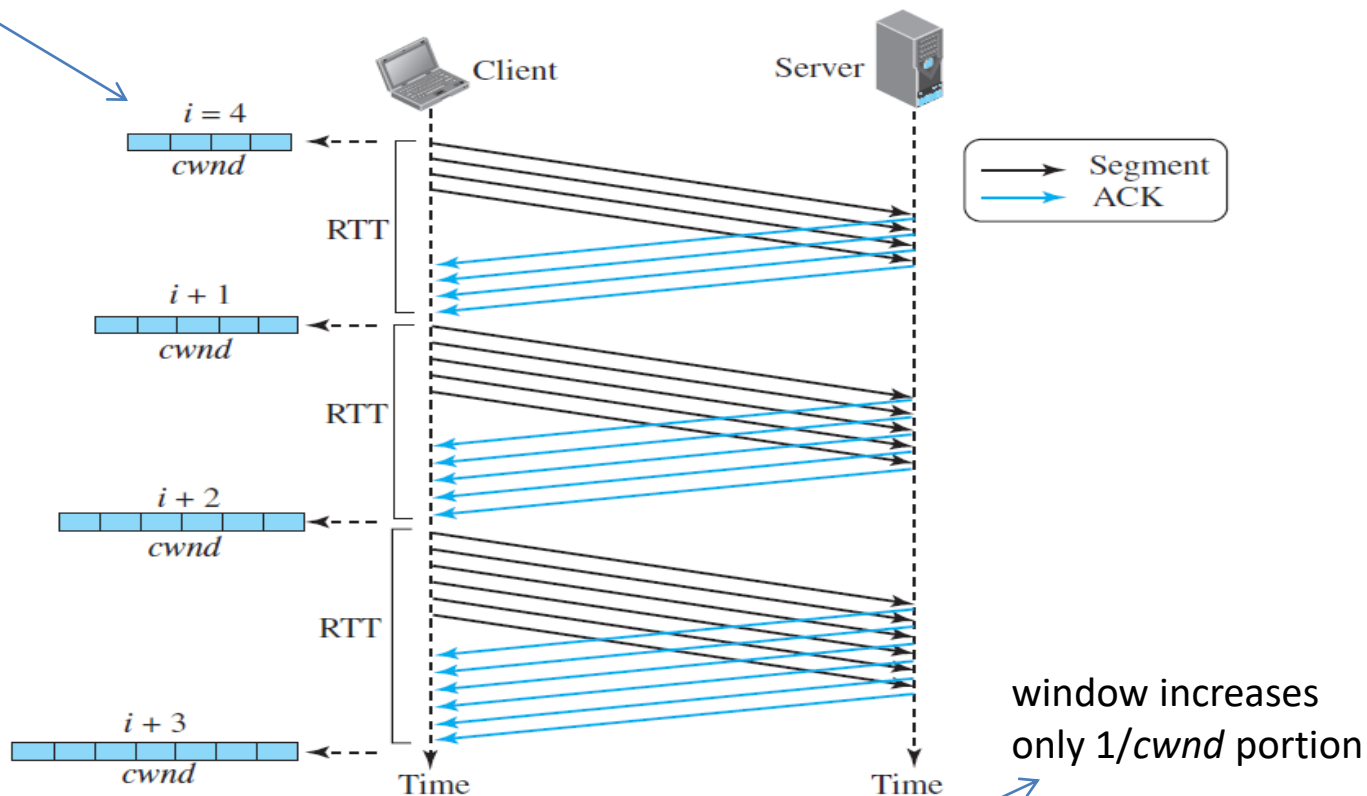
→ $cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Congestion Avoidance: Additive Increase

Figure 24.30 Congestion avoidance, additive increase

congestion window reaches the slow-start threshold



If an ACK arrives, $cwnd = cwnd + (1/cwnd)$.

Start
After 1 RTT
After 2 RTT
After 3 RTT

→ $cwnd = i$
→ $cwnd = i + 1$
→ $cwnd = i + 2$
→ $cwnd = i + 3$

In the congestion-avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

Fast Recovery:

The **fast-recovery** algorithm is optional in TCP. Only new versions try to use it

- It starts when three duplicate ACKs arrive, which is interpreted as light congestion in the network.
- Like congestion avoidance, this algorithm is also an additive increase, but it increases the size of the congestion window when a duplicate ACK arrives (after the three duplicate ACKs that trigger the use of this algorithm)

Three versions of TCP: **Tahoe TCP, Reno TCP, and New Reno TCP**

Taho TCP

- It used only two different algorithms in their congestion policy: ***slow start*** and ***congestion avoidance***
- Taho TCP treats the two signs used for congestion detection, **time-out** and **three duplicate ACKs**

Example 24.9

Figure 24.32 shows an example of congestion control in a Tahoe TCP.

TCP starts data transfer and sets the *ssthresh* variable to an ambitious value of 16 MSS. TCP begins at the slow-start (SS) state with the *cwnd* = 1. The congestion window grows exponentially, but a time-out occurs after the third RTT (before reaching the threshold). TCP assumes that there is congestion in the network. It immediately sets the new *ssthresh* = 4 MSS (half of the current *cwnd*, which is 8) and begins a new slow-start (SA) state with *cwnd* = 1 MSS. The congestion window grows exponentially until it reaches the newly set threshold. TCP now moves to the congestion-avoidance (CA) state and the congestion window grows additively until it reaches *cwnd* = 12 MSS. At this moment, three duplicate ACKs arrive, another indication of congestion in the network. TCP again halves the value of *ssthresh* to 6 MSS and begins a new slow-start (SS) state. The exponential growth of the *cwnd* continues. After RTT 15, the size of *cwnd* is 4 MSS. After sending four segments and receiving only two ACKs, the size of the window reaches the *ssthresh* (6) and TCP moves to the congestion-avoidance state. The data transfer now continues in the congestionavoidance (CA) state until the connection is terminated after RTT 20.

Tahoe TCP

Figure 24.32 Example 24.9

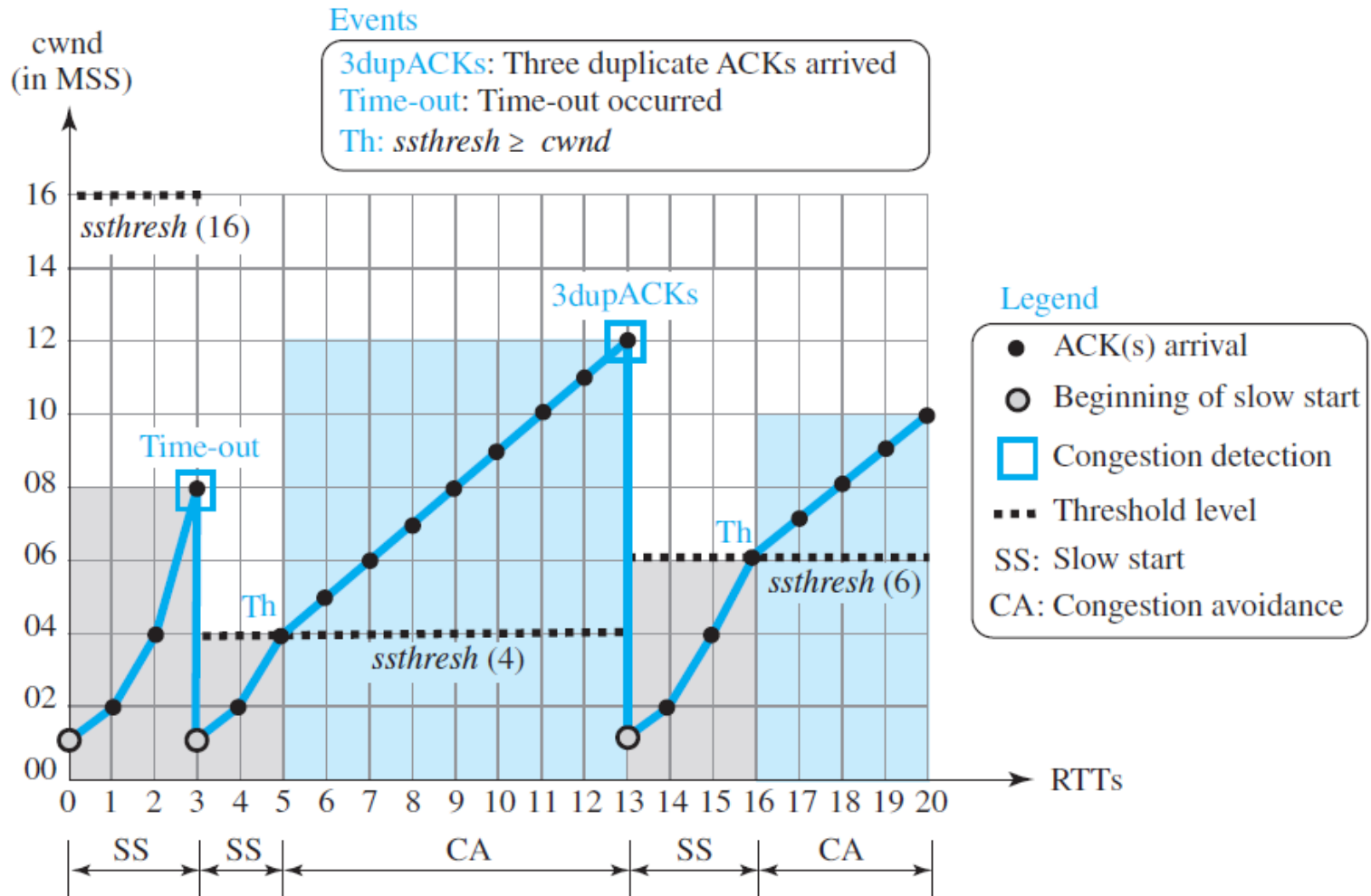
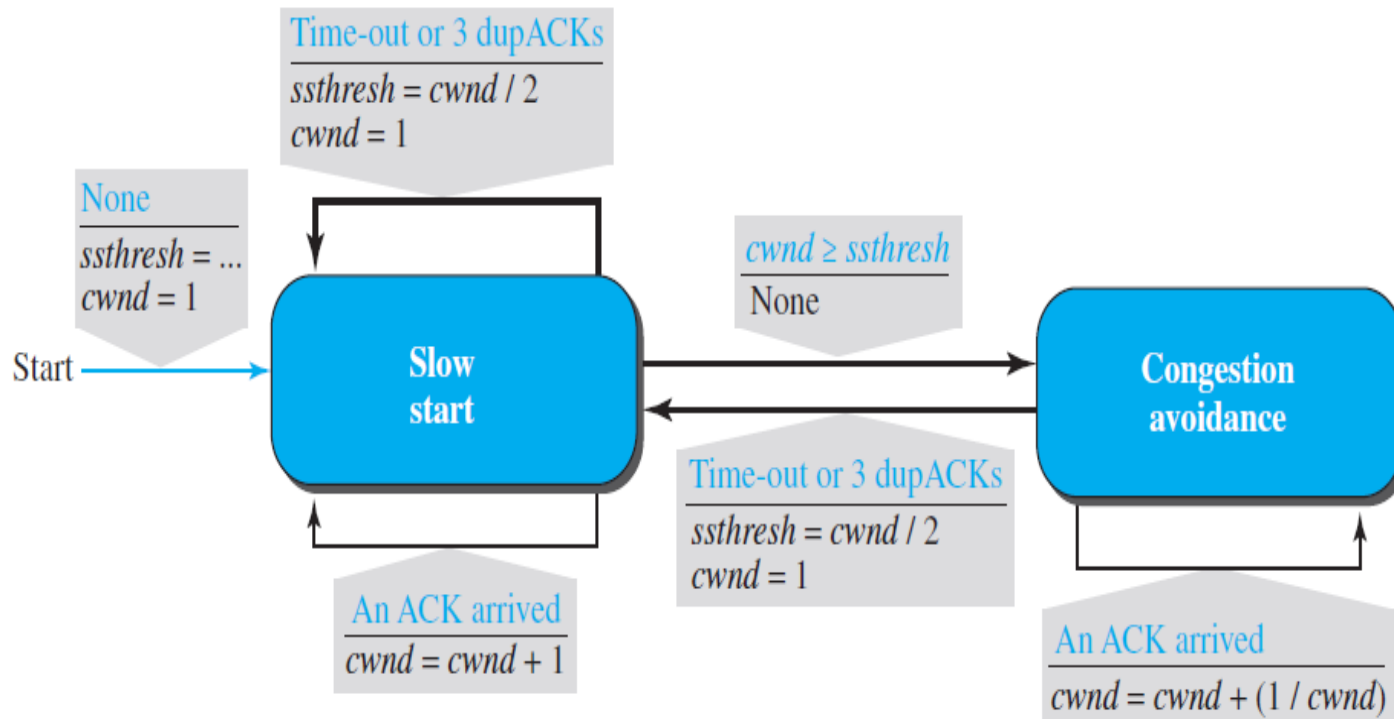


Figure 24.31 *FSM for Tahoe TCP*

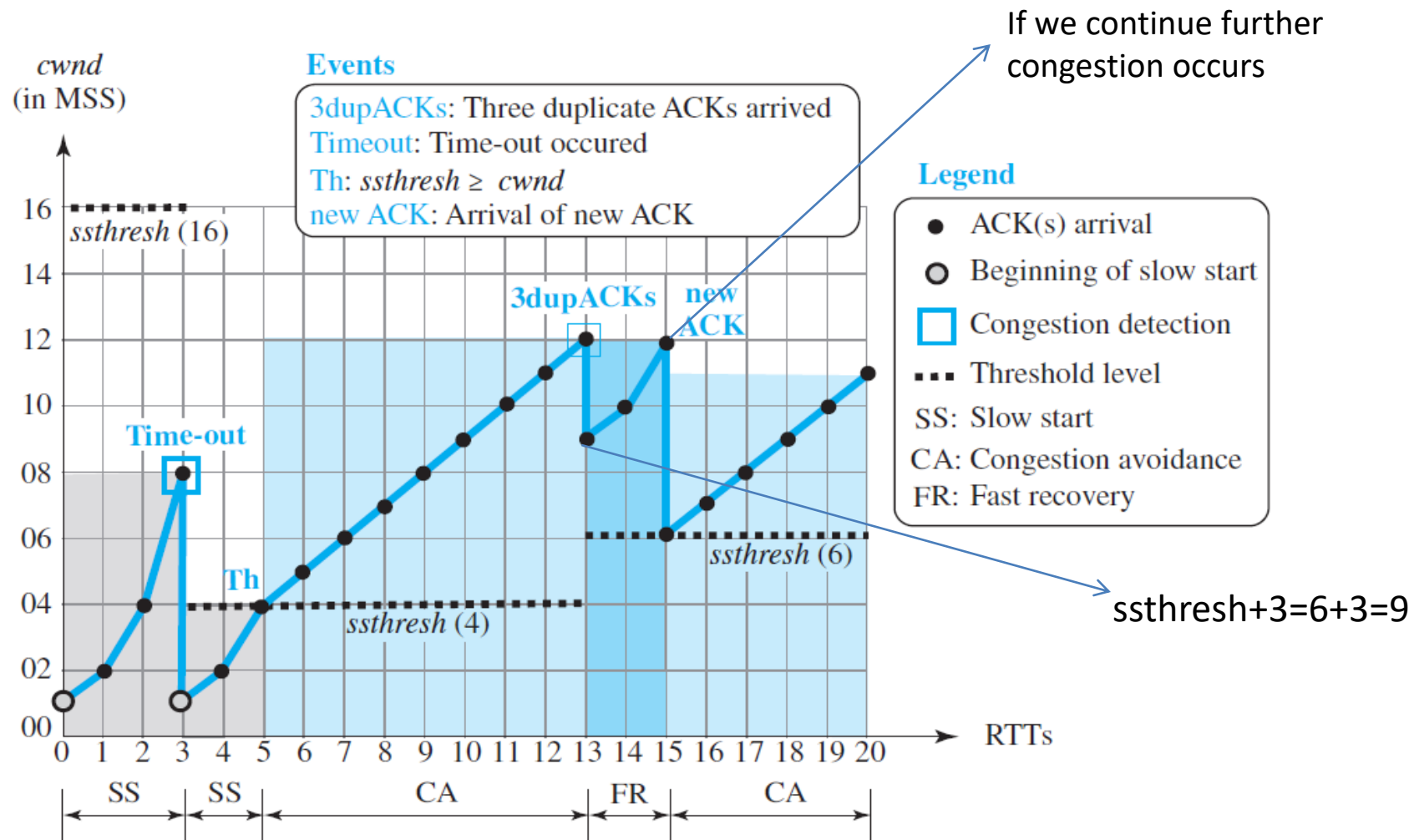


Reno TCP

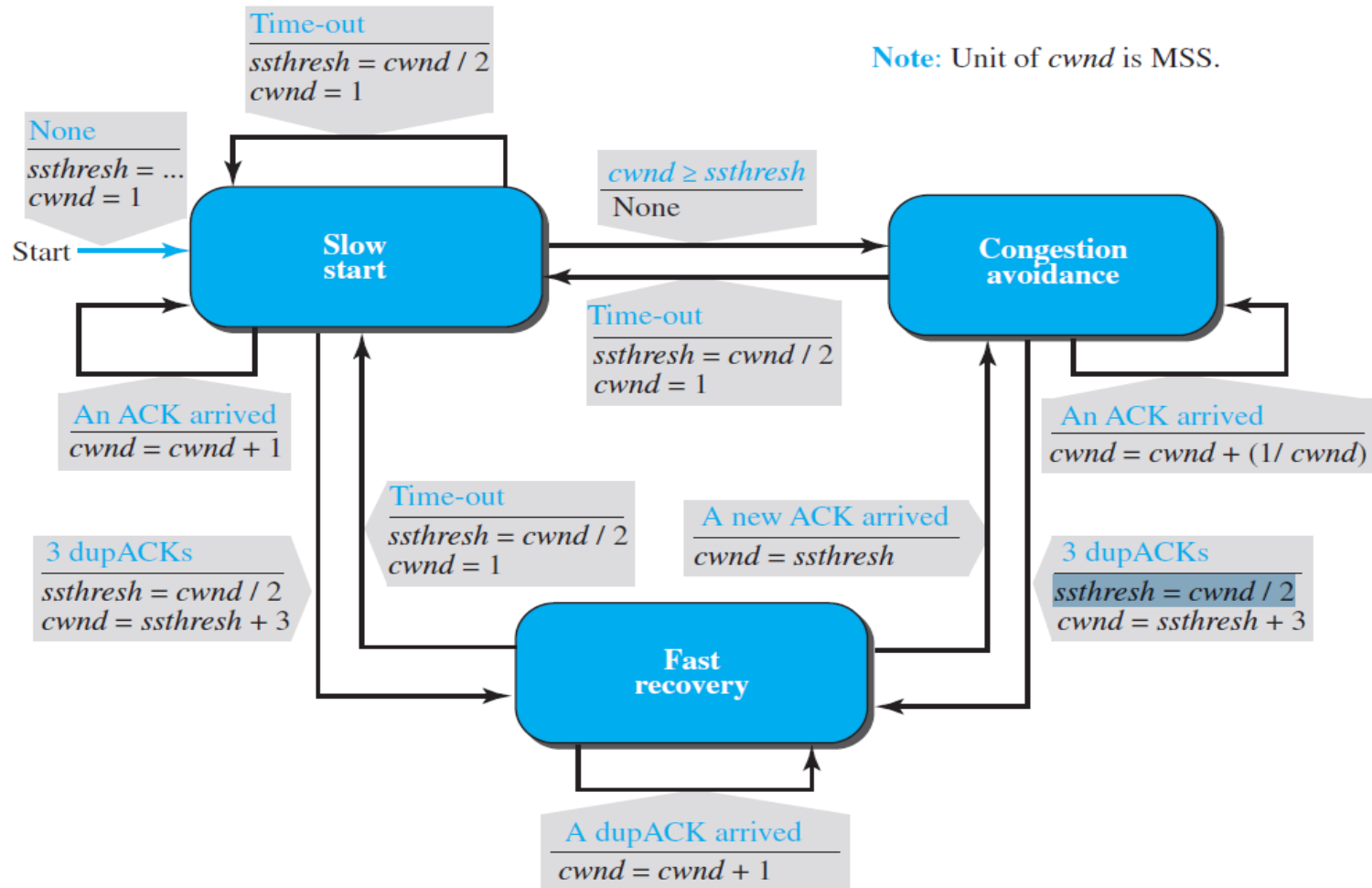
- It used three different algorithms in their congestion policy: ***slow start***, ***congestion avoidance*** and ***fast-recovery state***.
- This version treated the two signals of congestion detection **time-out** and the **arrival of three duplicate ACKs**
- If a time-out occurs, TCP moves to the slow-start state
- If three duplicate ACKs arrive, TCP moves to the fast-recovery state and remains there as long as more duplicate ACKs arrive
- fast-recovery state is a state somewhere between the slow-start and the congestion-avoidance states

Reno TCP

Figure 24.34 Example 24.10



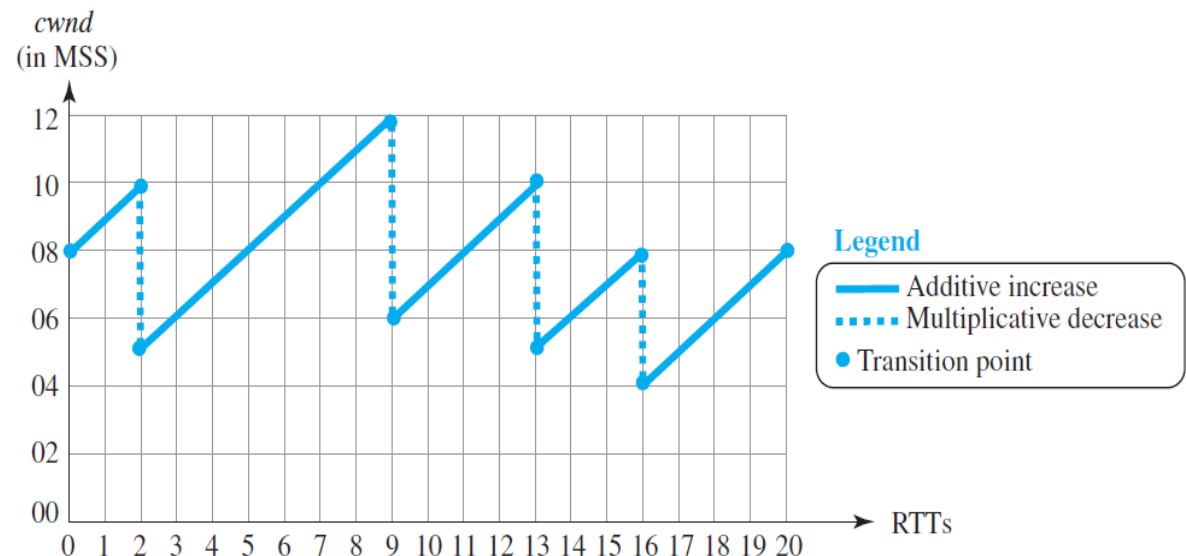
until timeout – no transmission



additive increase, multiplicative decrease (AIMD)-TCP Reno

- The Reno version is most common today
- In a long TCP connection, if we ignore the slow-start states and short exponential growth during fast recovery, the TCP congestion window is $cwnd = cwnd + (1 / cwnd)$ when an ACK arrives (congestion avoidance), and $cwnd = cwnd / 2$ when congestion is detected, as though SS does not exist and the length of FR is reduced to zero
- The congestion window size, after it passes the initial slow-start state, follows a saw tooth pattern called ***additive increase, multiplicative decrease (AIMD)***,

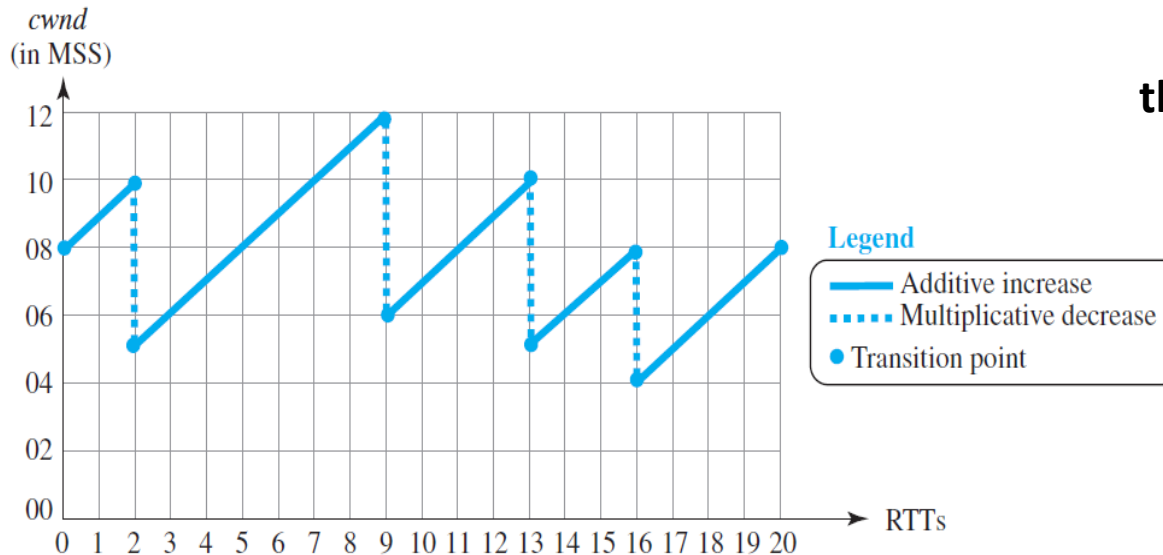
Figure 24.35 Additive increase, multiplicative decrease (AIMD)



TCP Throughput

If MSS = 10 KB (kilobytes) and RTT = 100 ms in Figure 24.35, we can calculate the throughput as shown below.

Figure 24.35 Additive increase, multiplicative decrease (AIMD)



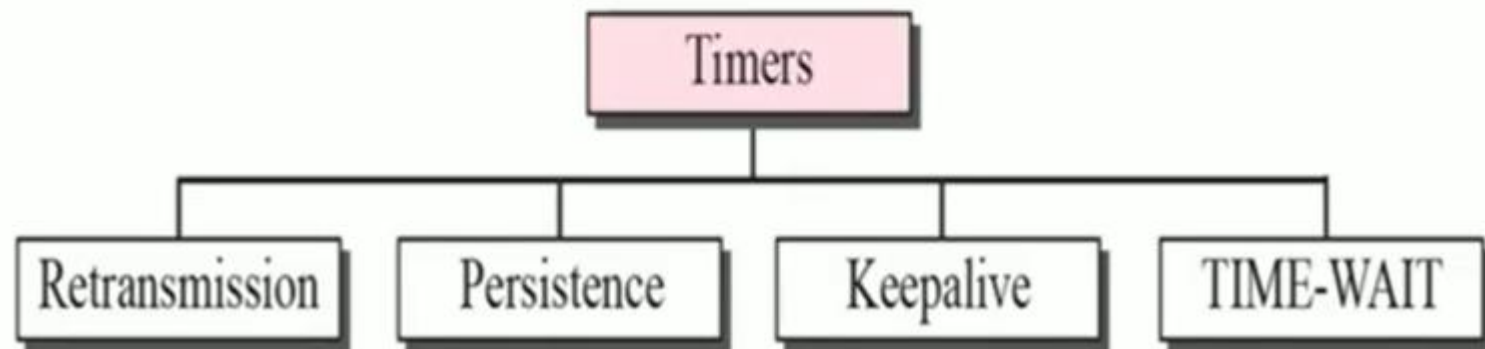
$$\text{throughput} = (0.75) W_{\max} / \text{RTT}$$

$W_{\max}$ is the average of window sizes when the congestion occurs

$$W_{\max} = (10 + 12 + 10 + 8 + 8) / 5 = 9.6 \text{ MSS}$$

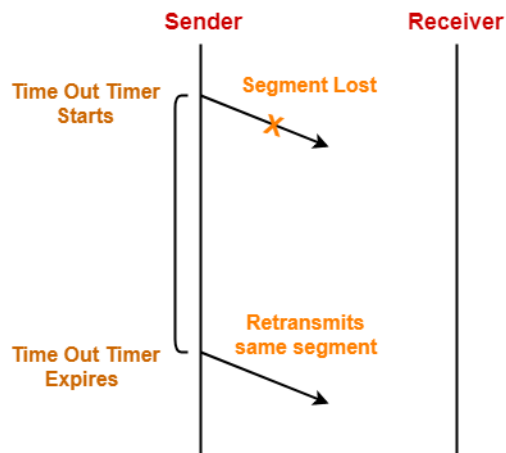
$$\text{Throughput} = (0.75 W_{\max} / \text{RTT}) = 0.75 \times 960 \text{ kbps} / 100 \text{ ms} = 7.2 \text{ Mbps}$$

TCP TIMERS



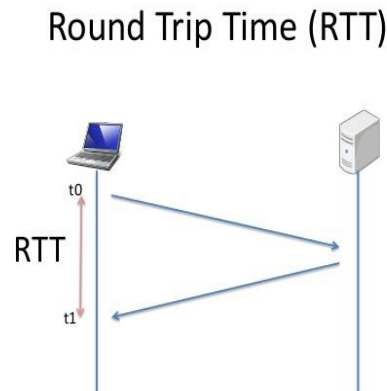
TCP Timers

- To reduce delay during communications, most TCP implementations use at least four timers: retransmission, persistence, keepalive, and TIME-WAIT.
- TCP employs one retransmission timer (for the whole connection period) that handles the retransmission time-out (RTO),

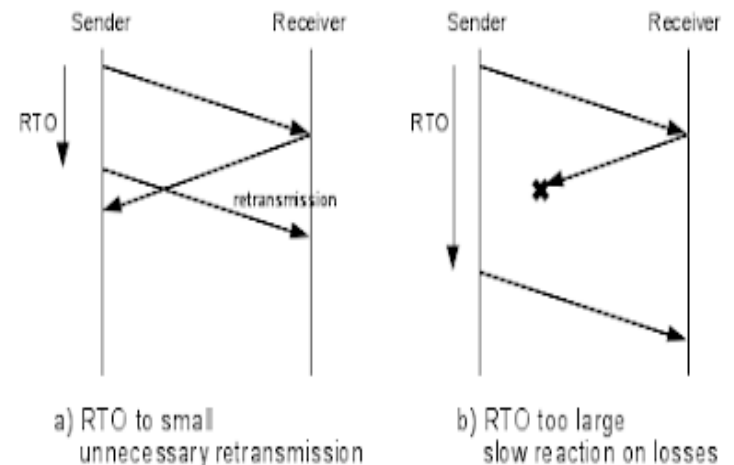


Retransmission after Time Out Timer Expiry

2015/11/10



Internet Initiative Japan Inc. (IIJ/AS2497)

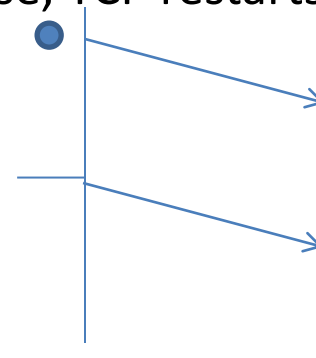


TCP Timers

Retransmission Timer : When TCP sends a segment the timer starts and stops when the acknowledge is received.

We can define the following rules for the retransmission timer:

1. When TCP sends the segment in front of the sending queue, it starts the timer.
2. When the timer expires, TCP resends the first segment in front of the queue, and restarts the timer.
3. When a segment or segments are cumulatively acknowledged, the segment or segments are purged from the queue.
4. If the queue is empty, TCP stops the timer; otherwise, TCP restarts the timer.



RTO- Round-Trip Time (RTT): RTT_M , RTT_S , RTT_D ,

- To calculate the retransmission time-out (RTO), we first need to calculate the **roundtrip time (RTT)**. Calculating RTT in TCP is an involved process that we explain step by step with some examples
- ***Measured RTT (RTT_M)***: round-trip time for a segment is the time required for the segment to reach the destination and be acknowledged.
- ***Smoothed RTT (RTT_S)***: The fluctuation is so high in today's Internet that a single measurement alone cannot be used for retransmission time-out purposes

Initially

→

No value

After first measurement

→

$RTT_S = RTT_M$

After each measurement

→

$RTT_S = (1 - \alpha) RTT_S + \alpha \times RTT_M$

α is implementation-dependent, but it is normally set to 1/8.

Round-Trip Time (RTT)

- **RTT Deviation:** Most implementations do not just use RTTs; they also calculate the RTT deviation

Initially

→

No value

After first measurement

→

$$RTT_D = RTT_M / 2$$

After each measurement

→

$$RTT_D = (1 - \beta) RTT_D + \beta \times |RTT_S - RTT_M|$$

The value of β is also implementation-dependent, but is usually set to 1/4

- **Retransmission Time-out (RTO):** The value of RTO is based on the smoothed roundtrip time and its deviation.

Original

→

Initial value

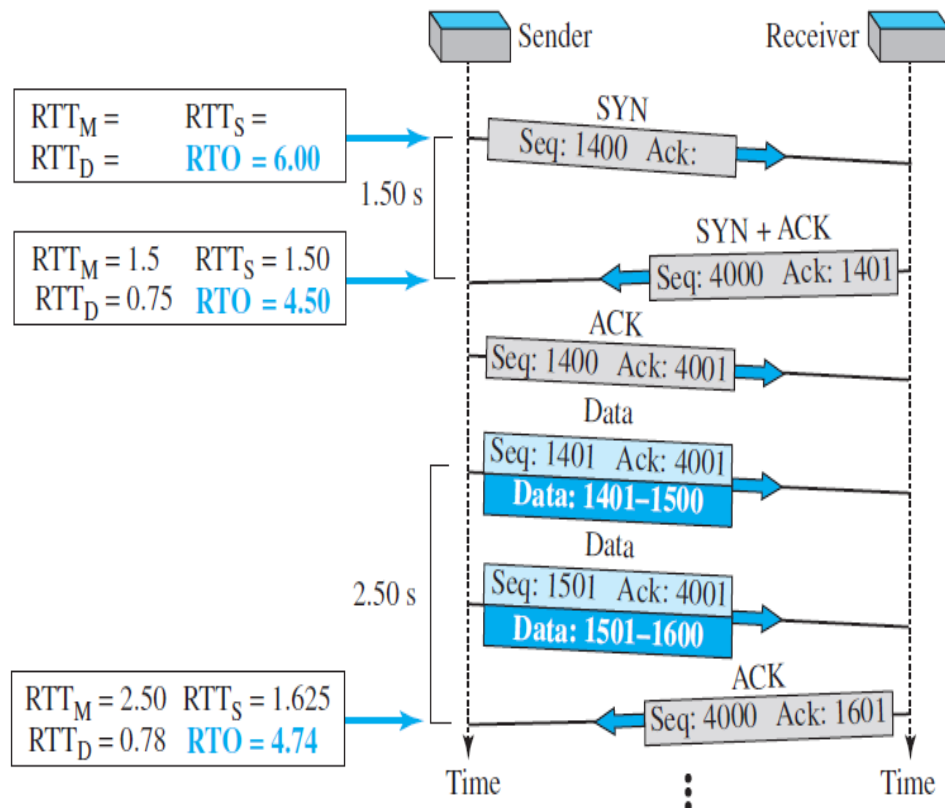
After any measurement

→

$$RTO = RTT_S + 4 \times RTT_D$$

Let us give a hypothetical example. Figure 24.36 shows part of a connection. The figure shows the connection establishment and part of the data transfer phases

Figure 24.36 Example 24.12



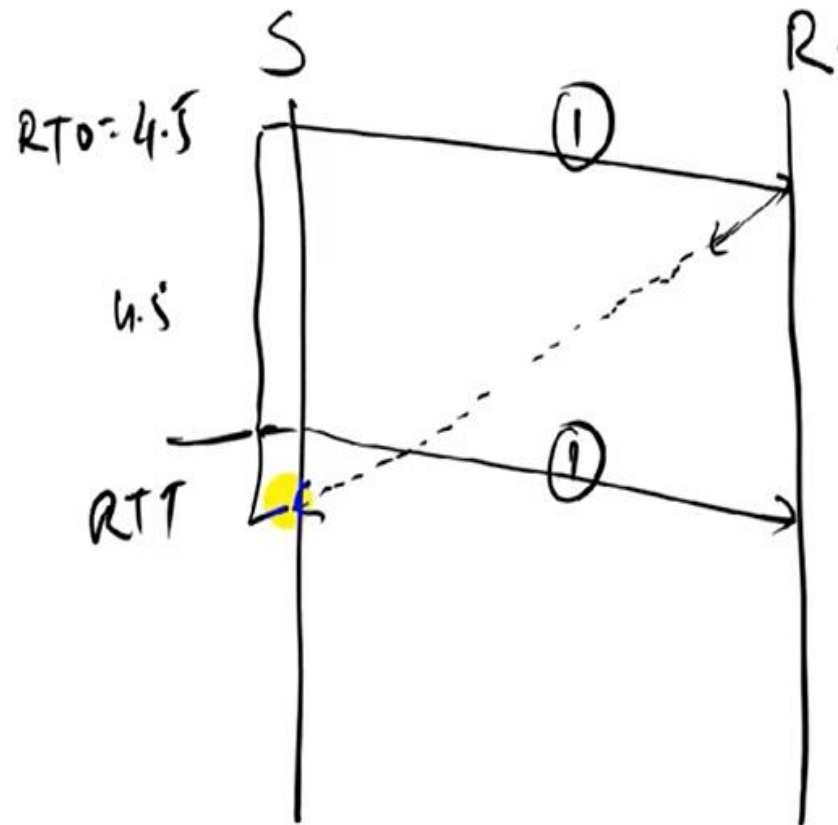
- When the SYN segment is sent, there is no value for RTT_M , RTT_S , or RTT_D . The value of RTO is set to 6.00 seconds
- When the SYN+ACK segment arrives

$$\begin{aligned}
 RTT_M &= 1.5 \\
 RTT_S &= 1.5 \\
 RTT_D &= (1.5)/2 = 0.75 \\
 RTO &= 1.5 + 4 \times 0.75 = 4.5
 \end{aligned}$$

$$\begin{aligned}
 RTT_M &= 2.5 \\
 RTT_S &= (7/8) \times (1.5) + (1/8) \times (2.5) = 1.625 \\
 RTT_D &= (3/4) \times (0.75) + (1/4) \times |1.625 - 2.5| = 0.78 \\
 RTO &= 1.625 + 4 \times (0.78) = 4.74
 \end{aligned}$$

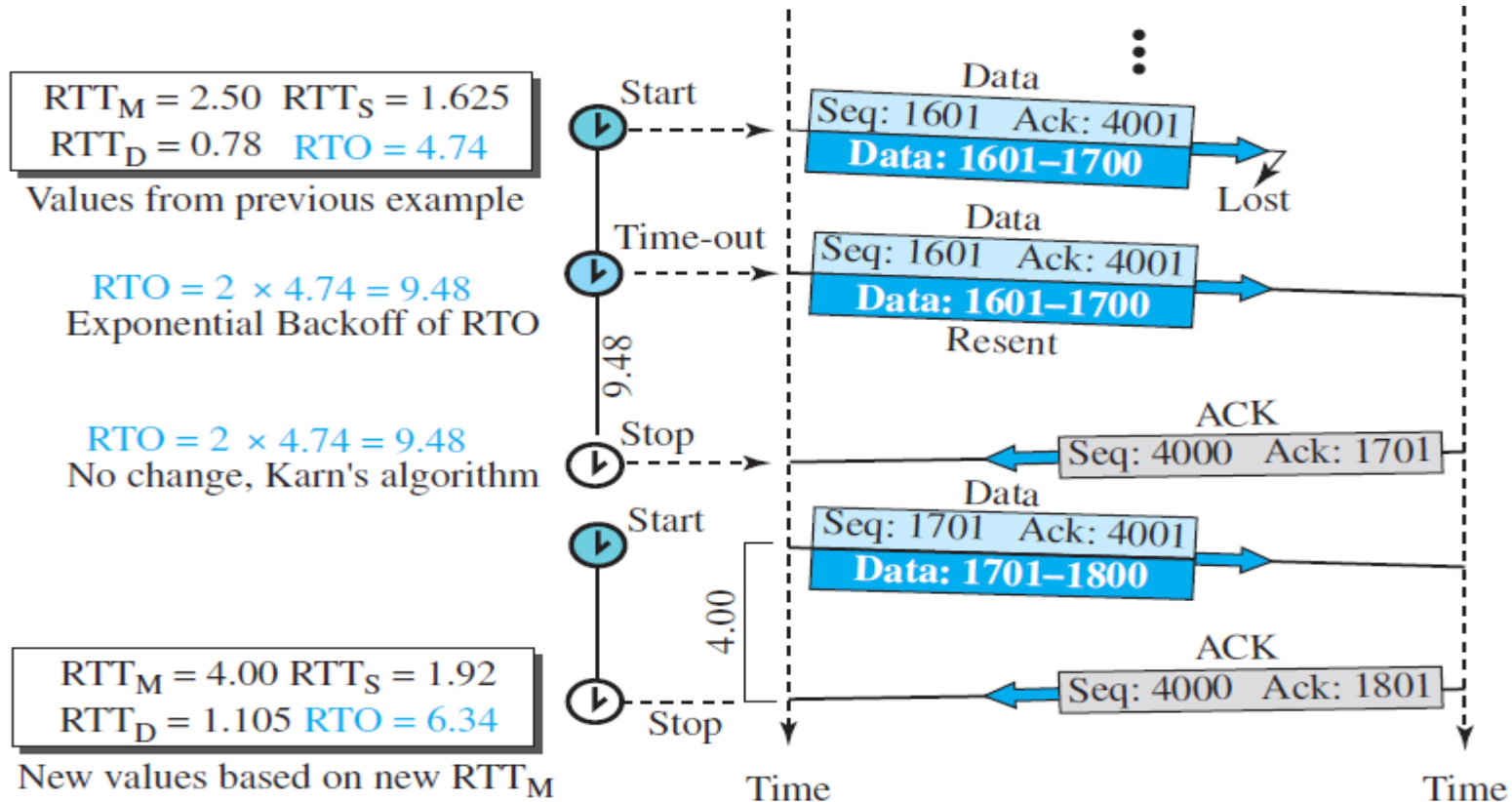
Karn's Algorithm

Problem



Karn's Algorithm

- Do not consider the **round-trip time of a retransmitted segment** in the calculation of RTTs.
- Do not update the value of RTTs until you send a segment and receive an acknowledgment without the need for retransmission.



Persistent Timer

- *Persistence timer* is designed to prevent deadlock
 - Receiver sends a packet with window size 0
 - Latter, it sends another packet with larger window size, letting the sender know that it can send data, but this segment gets lost
 - Both the receiver and transmitter are waiting for the other
 - Solution: persistence timer on the sender end, that goes off and produces a probe packet to go to the receiver and make it to advertise again its window

Keepalive Timer

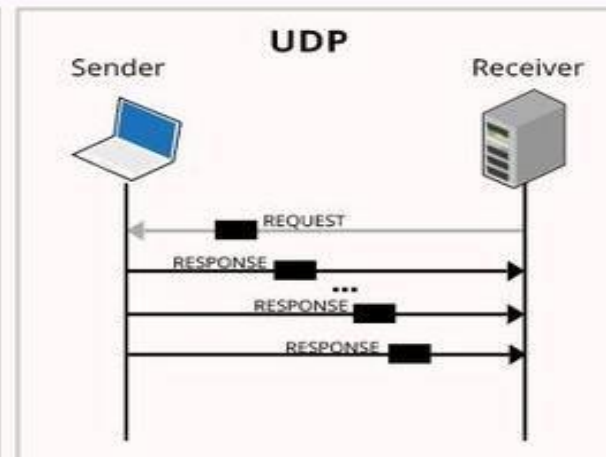
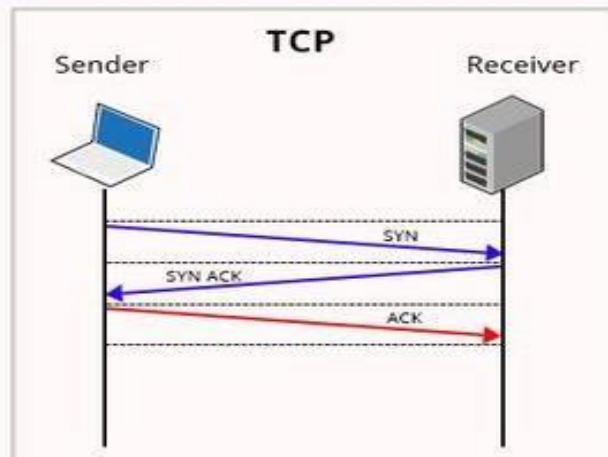
- A keepalive timer is used in some implementations to prevent a long idle connection between two TCPs.
- Suppose that a client opens a TCP connection to a server, transfers some data, and becomes silent. Perhaps the client has crashed.
- To avoid this, most implementations equip a server with a keepalive timer. If the server does not hear from the client after 2 hours, it sends a probe segment. If there is no response after 10 probes, each of which is 75 seconds apart, it assumes that the client is down and terminates the connection.

TIME-WAIT Timer

- The TIME-WAIT (2MSL) timer is used during connection termination.
- The 2MSL timer is used when TCP performs an active close and sends the final ACK. The connection must stay open for 2 MSL amount of time to allow TCP to resend the final ACK in case the ACK is lost.

TCP Vs. UDP

TCP	UDP
Connection Oriented Protocol	Connection-less Protocol
Connection in byte stream	Connection in message stream
It doesn't support multicasting and broadcasting	It supports broadcasting
It provides error control and flow control	Error Control and Flow control is not provided
Supports full Duplex	Does not support full Duplex
TCP packet is called as Segment	UDP packet is called as User Datagram



Sample questions

- **Define silly window syndrome? What are the solutions to avoid it (Nagles and Clarkes solution)?**
- Explain TCP flow control with an example.
- What are the mechanisms for congestion detection in TCP? Explain the general policy for handling congestion?
- **Explain TCP Timers.**
- Explain the Karns algorithm.
- TCP Tahoe and Reno

References

- Forouzan, A. Behrouz. *Data communications & networking (sie)*. Tata McGraw-Hill Education, 5E